

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

26 个 Kernel · 10 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 6 月 21 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (共 26 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler ×4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory → blocks/SM; block 大小 → blocks/SM
49
50 // ---- 常见优化手段速查清单 ----
51 //
52 // 1. Coalesced Memory Access (合并访问)
53 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
54 //   - 否则产生多次事务 (最坏 32 次)
55 //
56 // 2. Tiling (分块)
57 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
58 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
59 //
60 // 3. Vectorized Memory Access (向量化)
61 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数
62 //   - float4 = 128-bit, 单条指令加载 16 bytes
63 //
64 // 4. Thread Tile (寄存器分块)
65 //   - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
66 //   - 减少线程总数, 降低同步开销
67 //
68 // 5. Bank Conflict Avoidance
69 //   - Shared memory 有 32 banks × 4 bytes
70 //   - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
71 //   - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
72 //
73 // 6. Pipeline / Double Buffering (流水线)
74 //   - cp.async 异步拷贝下一批数据时同时做当前批的计算

```

```

75 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
76 //
77 // 7. Tensor Core (MMA / WGMMMA)
78 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
79 // - WGMMMA m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
80 //
81 // 8. Warp Specialization (Hopper+)
82 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
83 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
84 //
85 // 9. TMA (Tensor Memory Accelerator, Hopper+)
86 // - 硬件 DMA 引擎, 支持 2D/3D 寻址, 零寄存器开销
87 // - 配合 cp.async.bulk 实现异步数据搬运
88
89 // ---- Roofline 分析公式 ----
90 //
91 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
92 //
93 // GEMM (M=N=K=4096):
94 // FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPS
95 // Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
96 // AI ≈ 137G / 200M ≈ 685 FLOPS/Byte → compute-bound (远超 H100 ridge point:
97 // FP16 TC ≈ 295:1, FP32 ≈ 20:1)
98 //
99 // GEMV (M=4096, K=4096):
100 // FLOPs = 2 × M × K = 2 × 40962 ≈ 33 MFLOPS
101 // Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
102 // AI ≈ 33M / 67M ≈ 0.5 FLOPS/Byte → severely memory-bound
103 //
104 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPS/Byte → memory-bound
105
106 // =====
107 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
108 // =====
109 // 面试要点:
110 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
111 // memory
112 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
113 // 注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
114 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
115
116 #include <algorithm>
117 #include <cuda_fp16.h>
118 #include <cuda_runtime.h>
119 #include <float.h>
120 #include <stdio.h>
121 #include <stdlib.h>
122 #include <math.h>
123 #include <cublas_v2.h>
124
125 // WGMMMA/TMA headers (sm90a only)
126 #if defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900
127 #include <cuda/barrier>
128 #endif
129
130 #define WARP_SIZE 32
131 #define INT4(value) (reinterpret_cast<int4*>(&(value)))[0])
132 #define FLOAT4(value) (reinterpret_cast<float4*>(&(value)))[0])
133 #define HALF2(value) (reinterpret_cast<half2*>(&(value)))[0])
134
135 // =====
136 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
137 // =====
138
139 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
140 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
141 // 复杂度 O(logN), N=32 时仅需 5 步
142 // 模板参数: T=数据类型, kWarpSize=segment width (默认 32)
143 // kWarpSize 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
144 // 当 kWarpSize < 32 (如 FA 中 kWarpSize=4) 时, 只有同 segment 的 lane 参与通信
145 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
146 //
147 // 初始: 每个 lane 持有自己的值 v0..v7
148 // lane: 0 1 2 3 4 5 6 7

```

```

149 // val: v0 v1 v2 v3 v4 v5 v6 v7
150 //
151 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
152 //      ┌───────────┐
153 // 对:  (0,4) (1,5) (2,6) (3,7)
154 //
155 // lane: 0 1 2 3 4 5 6 7
156 // val: v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
157 //
158 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
159 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
160 // 对:  (0,2) (1,3) (4,6) (5,7)
161 //      ─── 前4个一组 ─── 后4个一组 ───
162 //
163 // lane: 0 1 2 3 4 5 6 7
164 // val: Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7} (每lane持有前一轮2个值的和再加本轮配对)
165 //      = v0+v2+v4+v6 ... (逐步归约)
166 //
167 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
168 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
169 // 对:  (0,1) (2,3) (4,5) (6,7)
170 //
171 // lane: 0 1 2 3 4 5 6 7
172 // val: Σall Σall Σall Σall Σall Σall Σall Σall ← 所有 lane 拥有全归约结果!
173 //
174 // mask=0: 循环终止, 归约完成。
175 //
176 // 关键性质:
177 // - XOR 配对是对称的: lane i 的配对对象是 lane i^mask, 而 (i^mask)^mask = i
178 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
179 // - 每轮信息传递距离减半: 16→8→4→2→1 (距离减半, 信息翻倍)
180 // - O(log2 N) 步完成, 无需 shared memory, 纯寄存器操作
181 // - __shfl_xor_sync 第四个参数 kWarpSize 限制 segment width:
182 //   当 kWarpSize=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
183 template <const int kWarpSize = WARP_SIZE, typename T = float>
184 __device__ __forceinline__ T warp_reduce_sum(T val) {
185 #pragma unroll
186     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
187         val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
188     }
189     return val;
190 }
191
192 // Warp Reduce Max — generic
193 template <const int kWarpSize = WARP_SIZE, typename T = float>
194 __device__ __forceinline__ T warp_reduce_max(T val) {
195 #pragma unroll
196     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
197         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpSize));
198     }
199     return val;
200 }
201
202 // =====
203 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
204 // =====
205
206 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
207 // 两级归约流程:
208 // 1. 每个 warp 内做 warp_reduce_sum → 得到每个 warp 的一个值
209 // 2. warp leader (lane=0) 写入 shared memory
210 // 3. syncthreads 后, lane 0~NUM_WARPS-1 读取 shared memory
211 // 4. 所有 warp 内再做一次 warp_reduce<NUM_WARPS> → 得到最终结果
212 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<NUM_WARPS 知道结果)
213 template <const int NUM_THREADS = 256>
214 __device__ float block_reduce_sum(float val) {
215     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
216     int warp = threadIdx.x / WARP_SIZE;
217     int lane = threadIdx.x % WARP_SIZE;
218     static __shared__ float shared[NUM_WARPS];
219
220     float value = warp_reduce_sum<WARP_SIZE>(val);
221     if (lane == 0)

```

```

222     shared[warp] = value;
223     __syncthreads();
224     value = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
225     value = warp_reduce_sum<NUM_WARPS>(value);
226     // 关键: broadcast 结果到所有线程, 后续用 result
227     // 做除法等操作时所有线程都能拿到
228     value = __shfl_sync(0xffffffff, value, 0, 32);
229     return value;
230 }
231
232 // Block Reduce Max — FP32 (增强版, 带 broadcast)
233 template <const int NUM_THREADS = 256>
234 __device__ float block_reduce_max(float val) {
235     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
236     int warp = threadIdx.x / WARP_SIZE;
237     int lane = threadIdx.x % WARP_SIZE;
238     static __shared__ float shared[NUM_WARPS];
239
240     float value = warp_reduce_max<WARP_SIZE>(val);
241     if (lane == 0)
242         shared[warp] = value;
243     __syncthreads();
244     value = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
245     value = warp_reduce_max<NUM_WARPS>(value);
246     value = __shfl_sync(0xffffffff, value, 0, 32);
247     return value;
248 }
249
250 // ---- Dot Product: y = sum(a[i] * b[i]) ----
251 // 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
252 // Grid: ((N + 127) / 128, 1, 1)
253 // Block: (128, 1, 1)
254 // source: LeetCode/kernels/dot-product/dot_product.cu
255 template <const int NUM_THREADS = 128>
256 __global__ void dot(float *a, float *b, float *y, int N) {
257     int tid = threadIdx.x;
258     int idx = blockIdx.x * NUM_THREADS + tid;
259     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
260     __shared__ float reduce_smem[NUM_WARPS];
261
262     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
263     int warp = tid / WARP_SIZE;
264     int lane = tid % WARP_SIZE;
265
266     prod = warp_reduce_sum<WARP_SIZE>(prod);
267     if (lane == 0)
268         reduce_smem[warp] = prod;
269     __syncthreads();
270
271     prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
272     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
273         prod = warp_reduce_sum<NUM_WARPS>(prod);
274     if (tid == 0)
275         atomicAdd(y, prod);
276 }
277
278 // Dot Product + float4
279 // Grid: ((N + 127) / 128, 1, 1)
280 // Block: (32, 1, 1), 128/4=32
281 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
282 // source: LeetCode/kernels/dot-product/dot_product.cu
283 template <const int NUM_THREADS = 128 / 4>
284 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
285     int tid = threadIdx.x;
286     int idx = (blockIdx.x * NUM_THREADS + tid) * 4;
287     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
288     __shared__ float reduce_smem[NUM_WARPS];
289
290     float4 reg_a = FLOAT4(a[idx]);
291     float4 reg_b = FLOAT4(b[idx]);
292     float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
293                             reg_a.z * reg_b.z + reg_a.w * reg_b.w)
294                       : 0.0f;
295     int warp = tid / WARP_SIZE;

```

```

296 int lane = tid % WARP_SIZE;
297
298 prod = warp_reduce_sum<WARP_SIZE>(prod);
299 if (lane == 0)
300     reduce_smem[warp] = prod;
301 __syncthreads();
302
303 prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
304 if (warp == 0)
305     prod = warp_reduce_sum<NUM_WARPS>(prod);
306 if (tid == 0)
307     atomicAdd(y, prod);
308 }
309
310 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
311 // 多 block 各自做 warp-smem-warp0 reduce, 然后 atomicAdd 到全局 y
312 // 跨 block 求和的常见模式, 适合 N 较大时使用;
313 // Grid: ((N + 127) / 128, 1, 1)
314 // Block: (128, 1, 1)
315 // source: LeetCode/kernels/reduce/block_all_reduce.cu
316 template <const int NUM_THREADS = 128>
317 __global__ void block_reduce_v2(float *a, float *y, int N) {
318     int tid = threadIdx.x;
319     int idx = blockIdx.x * NUM_THREADS + tid;
320     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
321     __shared__ float reduce_smem[NUM_WARPS];
322
323     float sum = (idx < N) ? a[idx] : 0.0f;
324     int warp = tid / WARP_SIZE;
325     int lane = tid % WARP_SIZE;
326
327     sum = warp_reduce_sum<WARP_SIZE>(sum);
328     if (lane == 0)
329         reduce_smem[warp] = sum;
330     __syncthreads();
331
332     sum = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
333     if (warp == 0)
334         sum = warp_reduce_sum<NUM_WARPS>(sum);
335     if (tid == 0)
336         atomicAdd(y, sum);
337 }
338
339 // =====
340 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
341 // =====
342 // 面试要点:
343 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
344 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
345 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
346
347 // ---- ReLU: y = max(0, x) ----
348 // Grid: ((N + 255) / 256, 1, 1)
349 // Block: (256, 1, 1)
350 // source: LeetCode/kernels/relu/relu.cu
351 __global__ void relu(float *x, float *y, int N) {
352     int idx = blockIdx.x * blockDim.x + threadIdx.x;
353     if (idx < N)
354         y[idx] = fmaxf(0.0f, x[idx]);
355 }
356
357 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
358 // block(64)×4(float4)=256 元素/block, 与基础版吞吐相同
359 // Grid: ((N + 255) / 256, 1, 1)
360 // Block: (64, 1, 1)
361 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
362 // source: LeetCode/kernels/relu/relu.cu
363 __global__ void relu_vec4(float *x, float *y, int N) {
364     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
365     if (idx < N) {
366         float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
367         float4 reg_y;
368         reg_y.x = fmaxf(0.0f, reg_x.x);
369         reg_y.y = fmaxf(0.0f, reg_x.y);

```

```

370     reg_y.z = fmaxf(0.0f, reg_x.z);
371     reg_y.w = fmaxf(0.0f, reg_x.w);
372     FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
373 }
374 }
375
376 // ---- Elementwise Add: c = a + b ----
377 // Grid: ((N + 255) / 256, 1, 1)
378 // Block: (256, 1, 1)
379 // source: LeetCUDA/kernels/elementwise/elementwise.cu
380 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
381     int idx = blockIdx.x * blockDim.x + threadIdx.x;
382     if (idx < N)
383         c[idx] = a[idx] + b[idx];
384 }
385
386 // Elementwise Add + float4 向量化
387 // Grid: ((N + 255) / 256, 1, 1)
388 // Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
389 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
390 // source: LeetCUDA/kernels/elementwise/elementwise.cu
391 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
392     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
393     if ((idx + 3) < N) {
394         float4 reg_a = FLOAT4(a[idx]);
395         float4 reg_b = FLOAT4(b[idx]);
396         float4 reg_c;
397         reg_c.x = reg_a.x + reg_b.x;
398         reg_c.y = reg_a.y + reg_b.y;
399         reg_c.z = reg_a.z + reg_b.z;
400         reg_c.w = reg_a.w + reg_b.w;
401         FLOAT4(c[idx]) = reg_c;
402     } else if (idx < N) {
403         for (int i = 0; (idx + i) < N; i++) {
404             c[idx + i] = a[idx + i] + b[idx + i];
405         }
406     }
407 }
408
409 // ---- Histogram: y[a[i]]++ ----
410 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
411 // Grid: ((N + 255) / 256, 1, 1)
412 // Block: (256, 1, 1)
413 // source: LeetCUDA/kernels/histogram/histogram.cu
414 __global__ void histogram(int *a, int *y, int N) {
415     int idx = blockIdx.x * blockDim.x + threadIdx.x;
416     if (idx < N)
417         atomicAdd(&y[a[idx]], 1);
418 }
419
420 // =====
421 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
422 // =====
423 // 面试要点:
424 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
425 //     online (1-pass, 增量更新)
426 //   - Online Softmax 是 FlashAttention 的数学基础
427 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
428 //     + variance)
429 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
430
431 // ---- Online Softmax 辅助结构 ----
432 // MD struct: 存储 running max (m) 和 running denominator (d)
433 // 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
434 // 核心递推公式:
435 //   m_new = max(m_old, x_i)
436 //   d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
437 struct __align__(8) MD {
438     float m; // running max
439     float d; // running denominator (sum of exp(x - max))
440 };
441
442 // Warp Reduce for Online Softmax
443 // 与普通 reduce 不同: 归约时需同时更新 m 和 d (因为 max 在不断变大)

```

```

444 template <const int kWarpSize = WARP_SIZE>
445 __device__ __forceinline__ MD warp_reduce_md_op(MD value) {
446     unsigned int mask = 0xffffffff;
447     #pragma unroll
448     for (int stride = kWarpSize >> 1; stride >= 1; stride >>= 1) {
449         MD other;
450         other.m = __shfl_xor_sync(mask, value.m, stride);
451         other.d = __shfl_xor_sync(mask, value.d, stride);
452
453         bool value_bigger = (value.m > other.m);
454         MD bigger_m = value_bigger ? value : other;
455         MD smaller_m = value_bigger ? other : value;
456
457         // 关键: 更新 d 时需要 rescale 旧的 d 到新的 max 尺度下
458         value.d = bigger_m.d + smaller_m.d * __expf(smaller_m.m - bigger_m.m);
459         value.m = bigger_m.m;
460     }
461     return value;
462 }
463
464 // =====
465 // Phase 3a: Softmax — 三级递进 (面试核心考点)
466 // =====
467 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点?」
468 // 回答线索: naive(溢出) → safe → online
469
470 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
471 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
472 // 问题: x 值很大时 exp(x) 溢出为 inf
473 template <const int NUM_THREADS = 256>
474 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
475 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
476 // source: LeetCode/kernels/softmax/softmax.cu
477 __global__ void softmax_per_token(float *x, float *y, int N) {
478     const int tid = threadIdx.x;
479     const int idx = blockIdx.x * blockDim.x + tid;
480
481     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
482     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
483     if (idx < N)
484         y[idx] = exp_val / exp_sum;
485 }
486
487 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
488 // 面试重点: 为什么 Softmax 需要 Safe?
489 // - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
490 // - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
491 // - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
492 // - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
493 // Grid: (S, 1, 1)
494 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
495 // source: LeetCode/kernels/softmax/softmax.cu
496 template <const int NUM_THREADS = 256>
497 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
498     const int tid = threadIdx.x;
499     const int idx = blockIdx.x * blockDim.x + tid;
500
501     // Pass 1: block reduce max — 找最大值
502     float val = (idx < N) ? x[idx] : -FLT_MAX;
503     float max_val = block_reduce_max<NUM_THREADS>(val);
504
505     // Pass 2: exp(x - max) → block reduce sum
506     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
507     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
508
509     if (idx < N)
510         y[idx] = exp_val / exp_sum;
511 }
512
513 // ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
514 // 面试重点: Online Softmax 为什么重要?
515 // - Safe Softmax 需要 3 遍遍历: 找 max → exp+sum → 除法
516 // - Online Softmax 用递推公式合并为 1 遍遍历
517 // - 核心思想: 处理新元素时, 用新旧 max 的差值 rescale 旧 denominator

```



```

518 // - 正是 FlashAttention 中 "online rescaling":
519 //   0_new = diag(exp(m_old - m_new)) * 0_old + exp(m_cur - m_new) * P@V
520 // 算法参考: "Online normalizer calculation for softmax" (arXiv:1805.02867)
521 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
522 // Grid: (S, 1, 1)
523 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
524 // source: LeetCUDA/kernels/softmax/softmax.cu
525 template <const int NUM_THREADS = 256>
526 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
527     int local_tid = threadIdx.x;
528     int global_tid = blockIdx.x * NUM_THREADS + threadIdx.x;
529     const int WARP_NUM = NUM_THREADS / WARP_SIZE;
530     int warp_id = local_tid / WARP_SIZE;
531     int lane_id = local_tid % WARP_SIZE;
532
533     // 初始化: 每个线程持有一个 (max, denom) 对
534     MD val;
535     val.m = global_tid < N ? x[global_tid] : -FLT_MAX;
536     val.d = global_tid < N ? 1.0f : 0.0f;
537
538     // Block reduce: warp_reduce_md_op 在归约中自动更新 m 和 d
539     __shared__ MD shared[WARP_NUM];
540     MD res = warp_reduce_md_op<WARP_SIZE>(val);
541
542     if (lane_id == 0)
543         shared[warp_id] = res;
544     __syncthreads();
545
546     // 第二级归约: warp0 收集各 warp 结果再做一次 md_op (复用 block_reduce 模式)
547     // 只有 local_tid < 32 的线程参与; shared[0] 在第二次 __syncthreads 后才对整个 block 可见
548     if (local_tid < WARP_SIZE) {
549         MD block_res =
550             local_tid < WARP_NUM ? shared[local_tid] : MD{-FLT_MAX, 0.0f};
551         block_res = warp_reduce_md_op<WARP_NUM>(block_res);
552         if (local_tid == 0) {
553             shared[0] = block_res;
554         }
555     }
556     __syncthreads();
557
558     // 用全局 max 和 denom 做最终 softmax
559     MD final_res = shared[0];
560     float d_total_inverse = __fdividef(1.0f, final_res.d);
561     // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
562     if (global_tid < N) {
563         y[global_tid] = __expf(x[global_tid] - final_res.m) * d_total_inverse;
564     }
565 }
566
567 // =====
568 // Phase 3b: RMS Normalization (1-pass reduce)
569 // =====
570 // 面试要点:
571 // - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
572 // - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
573 // - Llama 系列使用 RMS Norm
574 // - grid(N, K/K), block(K): 一行一个 block
575 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
576 // Block: (128, 1, 1), NUM_THREADS=K=128 (K>128 时调整模板参数)
577 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
578 template <const int NUM_THREADS = 128>
579 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
580     int tid = threadIdx.x;
581     int bid = blockIdx.x;
582     int idx = bid * blockDim.x + threadIdx.x;
583     const float epsilon = 1e-5f;
584
585     __shared__ float s_variance;
586     float value = (idx < N * K) ? x[idx] : 0.0f;
587     float variance = value * value;
588     variance = block_reduce_sum<NUM_THREADS>(variance);
589     if (tid == 0)
590         s_variance = rsqrtf(variance / ((float)K + epsilon)); // 1/rms(x)
591     __syncthreads();

```

```

592     if (idx < N * K)
593         y[idx] = (value * s_variance) * g;
594 }
595
596 // RMS Norm + float4
597 // Grid: (N, 1, 1)
598 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
599 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
600 // source: LeetCode/kernels/rms-norm/rms_norm.cu
601 template <const int NUM_THREADS = 128 / 4>
602 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
603     int tid = threadIdx.x;
604     int bid = blockIdx.x;
605     int idx = (bid * blockDim.x + threadIdx.x) * 4;
606     const float epsilon = 1e-5f;
607
608     __shared__ float s_variance;
609     float4 reg_x = FLOAT4(x[idx]);
610     float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
611                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
612                                     : 0.0f;
613     variance = block_reduce_sum<NUM_THREADS>(variance);
614     if (tid == 0)
615         s_variance = rsqrtf(variance / (float)K + epsilon);
616     __syncthreads();
617     float4 reg_y;
618     reg_y.x = reg_x.x * s_variance * g;
619     reg_y.y = reg_x.y * s_variance * g;
620     reg_y.z = reg_x.z * s_variance * g;
621     reg_y.w = reg_x.w * s_variance * g;
622     if (idx < N * K)
623         FLOAT4(y[idx]) = reg_y;
624 }
625
626 // =====
627 // Phase 3c: Layer Normalization (2-pass reduce)
628 // =====
629 // 面试要点:
630 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
631 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)^2)/K)
632 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
633 // Grid: (N, 1, 1), 一行一个 block
634 // Block: (128, 1, 1), NUM_THREADS=K=128
635 // source: LeetCode/kernels/layer-norm/layer_norm.cu
636 template <const int NUM_THREADS = 128>
637 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
638     int tid = threadIdx.x;
639     int bid = blockIdx.x;
640     int idx = bid * blockDim.x + threadIdx.x;
641     const float epsilon = 1e-5f;
642
643     __shared__ float s_mean;
644     __shared__ float s_variance;
645     float value = (idx < N * K) ? x[idx] : 0.0f;
646
647     // Pass 1: compute mean
648     float sum = block_reduce_sum<NUM_THREADS>(value);
649     if (tid == 0)
650         s_mean = sum / (float)K;
651     __syncthreads(); // 必须等待 s_mean 对所有线程可见
652
653     // Pass 2: compute variance = (x - mean)^2
654     float variance = (value - s_mean) * (value - s_mean);
655     variance = block_reduce_sum<NUM_THREADS>(variance);
656     if (tid == 0)
657         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
658     __syncthreads(); // 必须等待 s_variance 对所有线程可见
659
660     if (idx < N * K)
661         y[idx] = ((value - s_mean) * s_variance) * g + b;
662 }
663
664 // Layer Norm + float4
665 // Grid: (N, 1, 1)

```

```

666 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
667 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
668 // source: LeetCode/kernels/layer-norm/layer_norm.cu
669 template <const int NUM_THREADS = 128 / 4>
670 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N,
671                                int K) {
672     int tid = threadIdx.x;
673     int bid = blockIdx.x;
674     int idx = (bid * blockDim.x + threadIdx.x) * 4;
675     const float epsilon = 1e-5f;
676
677     __shared__ float s_mean;
678     __shared__ float s_variance;
679     float4 reg_x = FLOAT4(x[idx]);
680     float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
681
682     float sum = block_reduce_sum<NUM_THREADS>(value);
683     if (tid == 0)
684         s_mean = sum / (float)K;
685     __syncthreads();
686
687     float4 reg_x_hat;
688     reg_x_hat.x = reg_x.x - s_mean;
689     reg_x_hat.y = reg_x.y - s_mean;
690     reg_x_hat.z = reg_x.z - s_mean;
691     reg_x_hat.w = reg_x.w - s_mean;
692     float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
693                     reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
694     variance = block_reduce_sum<NUM_THREADS>(variance);
695     if (tid == 0)
696         s_variance = rsqrtf(variance / (float)K + epsilon);
697     __syncthreads();
698
699     float4 reg_y;
700     reg_y.x = reg_x_hat.x * s_variance * g + b;
701     reg_y.y = reg_x_hat.y * s_variance * g + b;
702     reg_y.z = reg_x_hat.z * s_variance * g + b;
703     reg_y.w = reg_x_hat.w * s_variance * g + b;
704     if (idx < N * K)
705         FLOAT4(y[idx]) = reg_y;
706 }
707
708 // =====
709 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
710 // =====
711 // 面试要点:
712 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
713 //     [x1'] [cos(θ) -sin(θ)] [x1]
714 //     [x2'] [sin(θ)  cos(θ)] [x2]
715 //   -  $\theta_i = 1 / (\theta^{(2i/d)})$ ,  $\theta=10000.0f$  (Llama 风格)
716 //   - token_pos = idx / N: token 在序列中的位置
717 //   - token_idx = idx % N: token 内的维度索引
718 //   - 输入 [seq_len, hidden_size], 输出同形状
719
720 // Grid: ((seq_len * N + 255) / 256, 1, 1)
721 // Block: (256, 1, 1)
722 // source: LeetCode/kernels/rope/rope.cu
723 __global__ void rope(float *x, float *out, int seq_len, int N) {
724     int idx = blockIdx.x * blockDim.x + threadIdx.x;
725     float x1 = x[idx * 2];
726     float x2 = x[idx * 2 + 1];
727     int token_pos = idx / N; // 序列位置
728     int token_idx = idx % N; // 维度索引
729
730     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
731     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
732     float sin_v = sinf(token_pos * exp_v);
733     float cos_v = cosf(token_pos * exp_v);
734
735     // 2D 旋转
736     float out1 = x1 * cos_v - x2 * sin_v;
737     float out2 = x1 * sin_v + x2 * cos_v;
738     out[idx * 2] = out1;
739     out[idx * 2 + 1] = out2;

```

```

740 }
741
742 // =====
743 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
744 // =====
745 // 面试要点 (Bank Conflict 专题) :
746 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
747 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
748 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
749 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
750 //
751 // 转置的四步演进:
752 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
753 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
754 //   BCF: smem 布局 [WARP_SIZE_S*4][WARP_SIZE_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict
755 //   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
756 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
757
758 // ---- Level 1: 基础版 (2D 索引, 合并读 + 非合并写) ----
759 // 每个线程处理 1 个元素, block(16,16)
760 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
761 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
762 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
763 // Block: (16, 16, 1)
764 // source: LeetCode/kernels/mat-transpose/mat_transpose.cu
765 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
766     const int global_x = blockIdx.x * blockDim.x + threadIdx.x;
767     const int global_y = blockIdx.y * blockDim.y + threadIdx.y;
768     if (global_y < row && global_x < col) {
769         // 读取: x[global_x][global_y] = x[global_x * col + global_y], 行优先, 合并
770         // 写入: y[global_y][global_x] = y[global_y * row +
771         // global_x], 列优先, 非合并
772         y[global_y * row + global_x] = x[global_x * col + global_y];
773     }
774 }
775
776 // ---- Level 4: 最佳版 (shared memory + bank conflict fix + merge write) ----
777
778 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
779 // Block: (16, 16, 1)
780 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
781 // source: LeetCode/kernels/mat-transpose/mat_transpose.cu
782 __global__ void mat_transpose_padded(
783     float *x, float *y, const int row, const int col) {
784     const int global_x = blockIdx.x * blockDim.x + threadIdx.x;
785     const int global_y = blockIdx.y * blockDim.y + threadIdx.y;
786     const int local_x = threadIdx.x;
787     const int local_y = threadIdx.y;
788
789     constexpr int WARP_SIZE_S = 16;
790     constexpr int PAD = 1;
791     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
792     __shared__ float tile[WARP_SIZE_S * 4][WARP_SIZE_S + PAD];
793
794     if (global_y * 4 < row && global_x < col) {
795         // Step 1: 从 global memory 读取 4 行, 写入 shared
796         // memory (行优先, 合并读取)
797         float4 x_val;
798         x_val.x = x[(global_y * 4) * col + global_x];
799         x_val.y = x[(global_y * 4 + 1) * col + global_x];
800         x_val.z = x[(global_y * 4 + 2) * col + global_x];
801         x_val.w = x[(global_y * 4 + 3) * col + global_x];
802         tile[local_y * 4][local_x] = x_val.x;
803         tile[local_y * 4 + 1][local_x] = x_val.y;
804         tile[local_y * 4 + 2][local_x] = x_val.z;
805         tile[local_y * 4 + 3][local_x] = x_val.w;
806         __syncthreads();
807
808         // Step 2: 从 shared memory 读取 ("转置"方向), 合并写入 global memory
809         float4 smem_val;
810         smem_val.x = tile[local_x * 4][local_y];
811         smem_val.y = tile[local_x * 4 + 1][local_y];
812         smem_val.z = tile[local_x * 4 + 2][local_y];
813         smem_val.w = tile[local_x * 4 + 3][local_y];

```

```

814     const int gid_x = blockIdx.x * blockDim.x;
815     const int gid_y = blockIdx.y * blockDim.y * 4;
816     const int out_y = gid_y + local_x * 4;
817     const int out_x = gid_x + local_y;
818
819     // 逐元素写回 (避免 reinterpret_cast<float4> 的对齐要求)
820     y[out_x * row + out_y + 0] = smem_val.x;
821     y[out_x * row + out_y + 1] = smem_val.y;
822     y[out_x * row + out_y + 2] = smem_val.z;
823     y[out_x * row + out_y + 3] = smem_val.w;
824 }
825 }
826 }
827
828 // =====
829 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
830 // =====
831 // 面试要点:
832 //   - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
833 //   - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
834 //   - 不同 K 值对应不同分块策略:
835 //       K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
836 //       K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
837 //       K=16 < 32: 一个 warp 用不满, 用 ROW_PER_WARP=2 让每个 warp 处理 2 行
838 //
839 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
840
841 // ---- SGEMV K32: 基础 warp-per-row ----
842 // 设计: block(32, 4), blockDim.x=WARP_SIZE=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 NUM_WARPS 次)
843 // grid(M/4), 每个 warp 负责一行
844 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
845 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
846 // Block: (32, 4, 1), 每行 1 warp
847 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
848 // source: LeetCode/kernels/sgemv/sgemv.cu
849 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
850     int tx = threadIdx.x; // 0~31
851     int ty = threadIdx.y; // 0~3
852     int bx = blockIdx.x;
853     int lane = tx % WARP_SIZE; // 0~31
854     int m = bx * blockDim.y + ty; // 全局行号
855     if (m < M) {
856         float sum = 0.0f;
857         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
858         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
859         const int NUM_ITERS = (K + WARP_SIZE - 1) / WARP_SIZE;
860 #pragma unroll
861         for (int w = 0; w < NUM_ITERS; ++w) {
862             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
863             int k = w * WARP_SIZE + lane;
864             sum += a[m * K + k] * x[k];
865         }
866         sum = warp_reduce_sum<WARP_SIZE>(sum);
867         // 每个 warp 处理一行, lane 0 写回结果
868         if (lane == 0)
869             y[m] = sum;
870     }
871 }
872
873 // ---- SGEMV K128: float4 向量化 ----
874 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
875 // Grid: ((M + 3) / 4, 1, 1)
876 // Block: (32, 4, 1)
877 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
878 // source: LeetCode/kernels/sgemv/sgemv.cu
879 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
880     int tx = threadIdx.x; // 0~31
881     int ty = threadIdx.y; // 0~3
882     int bx = blockIdx.x;
883     int lane = tx % WARP_SIZE; // 0~31
884     int m = blockDim.y * bx + ty;
885
886     if (m < M) {
887         float sum = 0.0f;

```

```

888 // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
889 const int NUM_ITERS = (((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
890 #pragma unroll
891 for (int w = 0; w < NUM_ITERS; ++w) {
892     int k = (w * WARP_SIZE + lane) * 4;
893     float4 reg_x = FLOAT4(x[k]);
894     float4 reg_a = FLOAT4(a[m * K + k]);
895     sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
896            reg_a.w * reg_x.w);
897 }
898 sum = warp_reduce_sum<WARP_SIZE>(sum);
899 if (lane == 0)
900     y[m] = sum;
901 }
902 }
903
904 // ---- SGEMV K16: K < WarpSize, ROW_PER_WARP=2 ----
905 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
906 // ROW_PER_WARP=2, K_WARP_SIZE=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
907 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
908 // Block: (32, 4, 1)
909 // 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理 2 行
910 // source: LeetCode/kernels/sgemv/sgemv.cu
911 template <const int ROW_PER_WARP = 2>
912 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
913     constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) / ROW_PER_WARP; // 16
914     int tx = threadIdx.x;
915     int ty = threadIdx.y;
916     int bx = blockIdx.x;
917     int lane = tx % WARP_SIZE;
918     int k = lane % K_WARP_SIZE; // 0~15
919     int m = (blockDim.y * bx + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
920     if (m < M) {
921         float sum = A[m * K + k] * x[k];
922         // 按照K_WARP_SIZE=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
923         sum = warp_reduce_sum<K_WARP_SIZE>(sum);
924         // 注意: 判断条件是 k == 0, 不是 lane == 0!
925         if (k == 0)
926             y[m] = sum;
927     }
928 }
929
930 // =====
931 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
932 // =====
933 // 面试要点 (GEMM 优化五层金字塔):
934 // Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
935 // Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
936 // 个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
937 // load/store 指令数 Level 4 — Tensor Core (MMA
938 // m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
939 // TMA (WGMMMA m64n128k16): Hopper 异步执行
940 //
941 // 计算密度递进:
942 // Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
943 // Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
944 // Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
945 //
946 // =====
947 // Phase 5a: SGEMM + HGEMM (非 Tensor Core 路径)
948 // =====
949
950 // ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
951 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
952 // C = A x B, C[M, N] = A[M, K] x B[K, N]
953 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
954 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
955 // Block: (32, 32, 1), 1024 线程
956 // source: LeetCode/kernels/sgemm/sgemm.cu
957 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
958     constexpr int BM = 32; // vec 版: 32×4 = 128
959     constexpr int BN = 32; // vec 版: 32×4 = 128
960     constexpr int BK = 32;
961     __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32×32×4=4KB smem, float = 4 bytes

```

```

962 int bx = blockIdx.x;
963 int by = blockIdx.y;
964 int tx = threadIdx.x;
965 int tid = threadIdx.y * blockDim.x + tx;
966
967 // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
968 int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载; vec 版: a_m = tid / (32 / 4), row 0~127, [128x32]
969 int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载; vec 版: a_k = (tid % (32 / 4)) * 4, t 0~7, col 0~31, 每个线程加载 4
970 个元素, 8x4 = 32
971 int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载; vec 版: b_k = tid / 32, row 0~31, [32x128]
972 int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载; vec 版: b_n = (tid % 32) * 4, t 0~31, col 0~127, 每个线程加载 4 个元
973 素, 32x4 = 128
974 int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row; vec 版: load_smem_a_m, 0~127, 0, 1, 2, ... (连续) [128x128]
975 int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col; vec 版: load_smem_b_n, 0~127, 0, 4, 8, ... (间隔)
976
977 float sum = 0.f; // 遍历完整的K, slice K; vec 版: sum[4][4] = 0.f; 每个线程处理4x4的大小, 才能保证32x32线程处理[32x4, 32x4]=[128
978 x128]大小
979 for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
980     int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
981     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
982     // vec 版: FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr])
983     s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
984     int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
985     int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
986     // vec 版: FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);
987     s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
988     __syncthreads(); // 确保整个 smem tile 加载完毕
989 }
990 #pragma unroll
991 for (int k = 0; k < BK; ++k) {
992     int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
993     int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
994     sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
995 }
996 __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
997 }
998 int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
999 int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1000 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1001 c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
1002 }
1003
1004 // ---- Level 1+: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
1005 // 在 Level 1 基础上引入两层优化:
1006 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1007 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1008 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1009 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major
1010 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4x4=16 个 C 元素
1011 // 1024 x 16 = 16384 = 128 x 128 ✓
1012 //
1013 // 线程到 4x4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1014 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1015 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1016 //
1017 // 加载映射 (每线程 4 个元素, float4):
1018 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8x4=32 列 ✓
1019 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32x4=128 列 ✓
1020 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1021 //
1022 // △ Bank Conflict 提示 (面试加分点):
1023 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1024 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict. 生产代码可用
1025 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1026 //
1027 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
1028 // Block: (32, 32, 1), 1024 线程
1029 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1030 // source: LeetCode/kernels/sgemm/sgemm.cu (vec4 variant)
1031 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1032     constexpr int BM = 128;
1033     constexpr int BN = 128;
1034     constexpr int BK = 32;

```



```

1033 __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1034 __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1035
1036 int bx = blockIdx.x;
1037 int by = blockIdx.y;
1038 int tx = threadIdx.x;
1039 int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1040
1041 // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1042 int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1043 int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1044 // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1045 int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1046 int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1047
1048 int load_gmem_a_m = by * BM + load_smem_a_m;
1049 int load_gmem_b_n = bx * BN + load_smem_b_n;
1050
1051 // 4x4 Thread Tile 基址 (独立于加载映射, 避免 /4 漏乘 bug)
1052 int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1053 int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1054
1055 float sum[4][4] = {0.f};
1056 for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1057     int load_gmem_a_k = bk * BK + load_smem_a_k;
1058     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1059     FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]);
1060     int load_gmem_b_k = bk * BK + load_smem_b_k;
1061     int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1062     FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);
1063     __syncthreads();
1064 }
1065 #pragma unroll
1066 for (int k = 0; k < BK; ++k) {
1067     // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4x4=16 次 FMA
1068     float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1069                        s_a[comp_smem_a_m_base + 1][k],
1070                        s_a[comp_smem_a_m_base + 2][k],
1071                        s_a[comp_smem_a_m_base + 3][k]};
1072     float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1073                        s_b[k][comp_smem_b_n_base + 1],
1074                        s_b[k][comp_smem_b_n_base + 2],
1075                        s_b[k][comp_smem_b_n_base + 3]};
1076 #pragma unroll
1077     for (int i = 0; i < 4; ++i) {
1078         #pragma unroll
1079         for (int j = 0; j < 4; ++j) {
1080             sum[i][j] += a_vals[i] * b_vals[j];
1081         }
1082     }
1083 }
1084 __syncthreads();
1085 }
1086
1087 // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1088 int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1089 int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1090 #pragma unroll
1091 for (int i = 0; i < 4; ++i) {
1092     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1093     float4 reg_c;
1094     reg_c.x = sum[i][0];
1095     reg_c.y = sum[i][1];
1096     reg_c.z = sum[i][2];
1097     reg_c.w = sum[i][3];
1098     FLOAT4(c[store_gmem_c_addr]) = reg_c;
1099 }
1100 }
1101
1102 // =====
1103 // Phase 5b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMA m64n128k16)
1104 // =====
1105 // 面试要点:
1106 // - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令

```



```

1107 // - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16×8] = [16×16]×[16×8]
1108 // - ldmatrix: 从 shared memory 加载 16×16 的矩阵片段到寄存器 (4 条 32-bit
1109 // 寄存器)
1110 // - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1111 // - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1112 //
1113 // ★ TN 布局详解 (面试高频考点):
1114 // TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1115 // BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1116 // N = Normal (列优先, BLAS 原生格式)
1117 // T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1118 // 第一个字母 → A 的 op, 第二个字母 → B 的 op
1119 // 所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1120 // BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1121 // 视角下: TN = A row-major [M×K], B^T row-major [N×K] (等价于 B col-major [K×N]) 在 cuBLAS
1122 // 调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1123 // T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1124 // N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1125 //
1126 // 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1127 // T = row-major (行优先, 对 BLAS 来说是 transposed)
1128 // N = col-major (列优先, BLAS native = normal)
1129 //
1130 // LeetCodeCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[M×N] = A[M×K] × B[K×N]
1131 // - A: row-major [M, K], B: row-major [K, N]
1132 // - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1133 // - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1134 //
1135 // TN 布局: C[M×N] = A[M×K] × B[K×N], B 以 B^T=[N×K] row-major 存储 (A 行优先, B 列优先)
1136 // - A [M×K]: row-major → 全局索引 A[m*K+k], smem s_a[BM][BK]
1137 // - B^T [N×K]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)
1138 // - 优势: B^T 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1139 // MMA 指令: mma.sync.aligned.m16n8k16.row.col
1140 // - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1141 //
1142 // WGMMA (Warp Group MMA, Hopper+):
1143 // - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1144 // - m64n128k16: 一次处理 64×128×16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1145 // - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1146 // threads) 做计算
1147 // - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
1148 //
1149 // =====
1150 // Phase 5b-1: MMA PTX 宏定义
1151 // =====
1152 //
1153 // ---- gmem → smem: cp.async ----
1154 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1155 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1156 #define CP_ASYNC_WAIT_GROUP(n) \
1157     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1158 //
1159 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1160 #define CP_ASYNC_CG(dst, src, bytes) \
1161     asm volatile( \
1162         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1163         "l"(src), "n"(bytes))
1164 //
1165 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1166 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1167 // 每次加载 8×8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1168 // aligned: 要求 128-bit 对齐
1169 // trans: 转置加载 (用于 col-major 的 B 矩阵)
1170 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1171     asm volatile( \
1172         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1173         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1174         : "r"(addr))
1175 //
1176 #define LDMATRIX_X2(R0, R1, addr) \
1177     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1178         : "=r"(R0), "=r"(R1) \
1179         : "r"(addr))
1180 //

```

```

1181 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1182 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1183 #define LDMATRIX_X2_T(R0, R1, addr)
1184     asm volatile(
1185         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n"
1186         : "=r"(R0), "=r"(R1)
1187         : "r"(addr))
1188
1189 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
1190 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1191 // row.col: A 是 row-major, B 是 col-major
1192 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1193 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1194 // C 矩阵大小 16×8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1195 #define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1)
1196     asm volatile(
1197         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, "
1198         "%4, %5}, {%6, %7}, {%8, %9};\n"
1199         : "=r"(RD0), "=r"(RD1)
1200         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1201         "r"(RC1))
1202
1203 // ---- MMA 辅助函数 ----
1204 // div_ceil: 整数除法向上取整
1205 #define HOST_DEVICE_INLINE __device__ __host__ inline
1206 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
1207     return (a % b != 0) ? (a / b + 1) : (a / b);
1208 }
1209
1210 // =====
1211 // Phase 5b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局
1212 // =====
1213 // 面试重点 — Tile Hierarchy:
1214 //   MMA Atom:      m16n8k16 (1 条 MMA 指令处理的最小 tile)
1215 //   MMA Tile (warp): 2×4=8 个 MMA atom = [32×32] (一个 warp 的计算)
1216 //   Warp Tile:     4×4=16 warps = [128×128] (整个 block 的计算)
1217 //   线程映射:      2×4=8 warps, 每个 warp 4×1=4 个 warp tile
1218 //   实际: MMA_TILE_M=2, MMA_TILE_N=4, WARP_TILE_M=4, WARP_TILE_N=4
1219 //       → BM=16×2×4=128, BN=8×4×4=128, Warps=2×4=8, Threads=8×32=256
1220 //
1221 // TN 布局在本 kernel 中的体现 (T=A 行优先, N=B 列优先):
1222 //   - A[M][K]: row-major, 全局索引 A[m*K + k], shared memory s_a[BM][BK]
1223 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major), 全局索引 B[n*K+k] 即原 B 元素 (k,n)
1224 //       (△ 内维连续的是 K) shared memory s_b[BN][BK] = s_b[N_tile][K_tile]
1225 //   - ldmatrix A: 用 x4 (非转置), 因为 A 是 row-major, ldmatrix 原生匹配
1226 //   - ldmatrix B: 用 x2 (非转置), 因为 B^T 在 smem 中为 row-major, ldmatrix 逐行加载
1227 //       B^T 的行即 B 的列, 天然匹配 MMA row.col 的 col-major B 输入, 无需 .trans
1228 //   - MMA 指令: row.col = A row, B col → 天然匹配 TN 布局
1229 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1230 //   - grid.z = S 个 swizzle 分区
1231 //   - grid.x = 每个分区内部需要发射多少个 128×128 的 N tiles
1232 //   - bx = blockIdx.z * gridDim.x + blockIdx.x, 把连续 block 打散到不同 N 区域, 改善 L2 命中
1233 // Block: (256, 1, 1), 8 warps
1234 // source: LeetCode/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1235 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
1236           const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1237           const int WARP_TILE_M = 4, const int WARP_TILE_N = 4,
1238           const int K_STAGE = 3, const bool BLOCK_SWIZZLE = false>
1239 __global__ void __launch_bounds__(256)
1240     hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1241     // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1242     const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1243     const int by = blockIdx.y;
1244     constexpr int BM = MMA_M * MMA_TILE_M * WARP_TILE_M; // 16*2*4=128
1245     constexpr int BN = MMA_N * MMA_TILE_N * WARP_TILE_N; // 8*4*4=128
1246     constexpr int BK = MMA_K; // 16
1247
1248     // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B
1249     // TN 布局: s_a[BM][BK]=[128][16] (A row-major), s_b[BN][BK]=[128][16] (B^T
1250     // row-major, 即 B col-major [K×N] 在 smem 中按 B^T[N×K] 存储)
1251     // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1252     // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1253     extern __shared__ half smem[];
1254     half *s_a = smem;

```

```

1255 half *s_b = smem + K_STAGE * BM * BK; // A 和 B 连续存放
1256 constexpr int s_a_stage_offset = BM * BK; // 128*16
1257 constexpr int s_b_stage_offset =
1258     BN * BK; // 128*16  $\Delta$  BN(128)xBK(16) = B^T row-major 的 smem 布局
1259
1260 const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1261 const int warp_id = tid / WARP_SIZE; // 0~7
1262 const int lane_id = tid % WARP_SIZE; // 0~31
1263 const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1264 const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1265
1266 // 线程到 global memory 的映射 (用于加载 A 和 B)
1267 // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1268 int load_smem_a_m = tid / 2; // 0~127
1269 int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1270 int load_smem_b_n = tid / 2; // 0~127  $\rightarrow$  B^T 的 N 方向 (row-major 的行)
1271 int load_smem_b_k =
1272     (tid % 2 == 0) ? 0 : 8; // 0, 8  $\rightarrow$  B^T 的 K 方向 (row-major 的列)
1273 int load_gmem_a_m = by * BM + load_smem_a_m;
1274 int load_gmem_b_n =
1275     bx * BN + load_smem_b_n; // B 全局列号 = N 方向的 tile 起始 + 线程偏移
1276 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1277     return;
1278
1279 // 累加器: 每个 warp 计算 WARP_TILE_MxWARP_TILE_N=16 个 MMA tile
1280 uint32_t RC[WARP_TILE_M][WARP_TILE_N][2] = {}; // 初始化为 0
1281
1282 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1283 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1284 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1285
1286 // 预加载前 (K_STAGE-1) 个 stage
1287 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [KxN])
1288 #pragma unroll
1289 for (int k = 0; k < (K_STAGE - 1); ++k) {
1290     int load_gmem_a_k = k * BK + load_smem_a_k;
1291     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1292     int load_gmem_b_k = k * BK + load_smem_b_k;
1293     int load_gmem_b_addr =
1294         load_gmem_b_n * K + load_gmem_b_k; // B^T: [n][k] row-major (即 B[k][n] col-major)  $\Delta$ 
1295
1296     uint32_t load_smem_a_ptr =
1297         (smem_a_base_ptr +
1298          (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) *
1299           sizeof(half));
1300     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1301
1302     uint32_t load_smem_b_ptr =
1303         (smem_b_base_ptr +
1304          (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) *
1305           sizeof(half));
1306     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1307
1308     CP_ASYNC_COMMIT_GROUP();
1309 }
1310
1311 const int NUM_K_TILES = div_ceil(K, BK);
1312 CP_ASYNC_WAIT_GROUP(K_STAGE - 2); // 等待前 (K_STAGE-2) 个 group 完成
1313 __syncthreads();
1314
1315 // 主循环: K 维分块迭代
1316 #pragma unroll
1317 for (int k = (K_STAGE - 1); k < NUM_K_TILES; ++k) {
1318     // Stage 选择: 轮转方式 (round-robin)
1319     // 这里统一用 %K_STAGE, 而不是只在 s2/s4 时写成 &1 / &3:
1320     // 原始实现要兼容 s3 这类非 2 的幂 stage 数, 因此直接用 mod 最稳妥。
1321     int smem_sel = (k + 1) % K_STAGE; // 当前计算的 stage
1322     int smem_sel_next = k % K_STAGE; // 下一轮加载的 stage
1323
1324     // 异步加载下一批数据到 smem_sel_next
1325     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用
1326     // n*K+k (row-major, 内维连续的是 K)
1327     int load_gmem_a_k = k * BK + load_smem_a_k;
1328     int load_gmem_a_addr =

```

```

1329     load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1330     int load_gmem_b_k = k * BK + load_smem_b_k;
1331     int load_gmem_b_addr =
1332         load_gmem_b_n * K + load_gmem_b_k; // B^T: row-major [n][k], 内维连续的是 K Δ
1333
1334     uint32_t load_smem_a_ptr =
1335         (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1336             load_smem_a_m * BK + load_smem_a_k) *
1337             sizeof(half));
1338     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1339
1340     uint32_t load_smem_b_ptr =
1341         (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1342             load_smem_b_n * BK + load_smem_b_k) *
1343             sizeof(half));
1344     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1345     CP_ASYNC_COMMIT_GROUP();
1346
1347     // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1348     // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major
1349     // B 用 x2 (非转置), smem 中 B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1350     uint32_t RA[WARP_TILE_M][4];
1351     uint32_t RB[WARP_TILE_N][2];
1352
1353     // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1354     #pragma unroll
1355     for (int i = 0; i < WARP_TILE_M; ++i) {
1356         int warp_smem_a_m = warp_m * (MMA_M * WARP_TILE_M) + i * MMA_M;
1357         int lane_smem_a_m =
1358             warp_smem_a_m + lane_id % 16; // ldmatrix 用 16 个 lane
1359         int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8
1360         uint32_t lane_smem_a_ptr =
1361             (smem_a_base_ptr +
1362                 (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1363                 sizeof(half));
1364         LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1365     }
1366
1367     // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1368     // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1369     // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1370     #pragma unroll
1371     for (int j = 0; j < WARP_TILE_N; ++j) {
1372         int warp_smem_b_n = warp_n * (MMA_N * WARP_TILE_N) + j * MMA_N;
1373         int lane_smem_b_n =
1374             warp_smem_b_n + lane_id % 8; // ldmatrix B 用 8 个 lane
1375         int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // 0, 8
1376         uint32_t lane_smem_b_ptr =
1377             (smem_b_base_ptr +
1378                 (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1379                 sizeof(half));
1380         LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1381     }
1382
1383     // MMA compute: 发射 WARP_TILE_M × WARP_TILE_N 条 MMA 指令
1384     #pragma unroll
1385     for (int i = 0; i < WARP_TILE_M; ++i) {
1386         #pragma unroll
1387         for (int j = 0; j < WARP_TILE_N; ++j) {
1388             HMMA16816(RC[i][j][0], RC[i][j][1], RA[i][0], RA[i][1], RA[i][2],
1389                 RA[i][3], RB[j][0], RB[j][1], RC[i][j][0], RC[i][j][1]);
1390         }
1391     }
1392
1393     // 等待当前 stage 的异步加载完成, 然后 sync
1394     CP_ASYNC_WAIT_GROUP(K_STAGE - 2);
1395     __syncthreads();
1396 }
1397
1398 // 尾端处理: 最后 (K_STAGE-1) 个 stage 的计算 (无新数据加载)
1399 if ((K_STAGE - 2) > 0) {
1400     CP_ASYNC_WAIT_GROUP(0);
1401     __syncthreads();
1402 }

```

```

1403 {
1404 #pragma unroll
1405 for (int k = 0; k < (K_STAGE - 1); k++) {
1406     uint32_t RA[WARP_TILE_M][4];
1407     uint32_t RB[WARP_TILE_N][2];
1408     int stage_sel = ((NUM_K_TILES - (K_STAGE - 1) + k) % K_STAGE);
1409
1410 #pragma unroll
1411 for (int i = 0; i < WARP_TILE_M; ++i) {
1412     int warp_smem_a_m = warp_m * (MMA_M * WARP_TILE_M) + i * MMA_M;
1413     int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1414     int lane_smem_a_k = (lane_id / 16) * 8;
1415     uint32_t lane_smem_a_ptr =
1416         (smem_a_base_ptr + (stage_sel * s_a_stage_offset +
1417             lane_smem_a_m * BK + lane_smem_a_k) *
1418             sizeof(half));
1419     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1420 }
1421 #pragma unroll
1422 for (int j = 0; j < WARP_TILE_N; ++j) {
1423     int warp_smem_b_n = warp_n * (MMA_N * WARP_TILE_N) + j * MMA_N;
1424     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1425     int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
1426     uint32_t lane_smem_b_ptr =
1427         (smem_b_base_ptr + (stage_sel * s_b_stage_offset +
1428             lane_smem_b_n * BK + lane_smem_b_k) *
1429             sizeof(half));
1430     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1431 }
1432 #pragma unroll
1433 for (int i = 0; i < WARP_TILE_M; ++i) {
1434 #pragma unroll
1435 for (int j = 0; j < WARP_TILE_N; ++j) {
1436     HMMA16816(RC[i][j][0], RC[i][j][1], RA[i][0], RA[i][1], RA[i][2],
1437         RA[i][3], RB[j][0], RB[j][1], RC[i][j][0], RC[i][j][1]);
1438 }
1439 }
1440 }
1441 }
1442 }
1443
1444 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1445 {
1446     for (int i = 0; i < WARP_TILE_M; ++i) {
1447         uint32_t RC0[WARP_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1448         uint32_t RC1[WARP_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1449         // 用 warp shuffle 收集同一个 MMA tile 内不同 lane 的结果。
1450         // 对 m16n8k16 的 C fragment:
1451         //   - lane 0/4/8/.../28 各自持有某一行里的 {c0,c1}
1452         //   - lane+1 持有同一行里的 {c2,c3}
1453         //   - lane+2 持有同一行里的 {c4,c5}
1454         //   - lane+3 持有同一行里的 {c6,c7}
1455         // 所以每个 4-lane 子组可以通过 shfl 收齐一整行 8 个 half, 然后由 lane%4==0
1456         // 的那个线程做一次 128-bit store。
1457 #pragma unroll
1458 for (int j = 0; j < WARP_TILE_N; ++j) {
1459     RC0[j][0] = RC[i][j][0];
1460     RC1[j][0] = RC[i][j][1];
1461     RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1462     RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1463     RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1464     RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1465     RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1466     RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1467 }
1468 // 每 4 个 lane 中只有 lane 0 做 128-bit store
1469 if (lane_id % 4 == 0) {
1470     int store_warp_smem_c_m = warp_m * (MMA_M * WARP_TILE_M) + i * MMA_M;
1471     int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1472 #pragma unroll
1473 for (int j = 0; j < WARP_TILE_N; ++j) {
1474     int store_warp_smem_c_n = warp_n * (MMA_N * WARP_TILE_N) + j * MMA_N;
1475     int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1476     int store_gmem_c_addr_0 =

```

```

1477         store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1478     int store_gmem_c_addr_1 =
1479         (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1480     // 128-bit store: 一次写入 8 个 half
1481     *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1482         *reinterpret_cast<float4*>(&RC0[j][0]);
1483     *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1484         *reinterpret_cast<float4*>(&RC1[j][0]);
1485     }
1486 }
1487 }
1488 }
1489 }
1490
1491 // =====
1492 // Phase 5b-3: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
1493 // =====
1494 // 面试要点 (WGMMMA vs MMA 对比):
1495 //   - MMA: warp 级 (32 threads), 同步执行
1496 //   - WGMMMA: warpgroup 级 (128 threads = 4 warps), 异步执行 (fire-and-forget)
1497 //   - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA 的 32
1498 //     倍)
1499 //   - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
1500 //   - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过 barrier
1501 //     同步
1502 //   - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
1503
1504 #if defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900
1505 // ---- WGMMMA 辅助函数 ----
1506 #define WGMMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
1507 #define WGMMMA_COMMIT_GROUP() \
1508     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
1509 #define WGMMMA_WAIT_GROUP(n) \
1510     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(n) : "memory")
1511
1512 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMMA descriptor 位域中的 14-bit 值。
1513 // 编码规则 (PTX ISA §9.7.15.5.1.2.2):
1514 //   encoded = (x & 0x3FFFF) >> 4
1515 // 其中 0x3FFFF = 2^18 - 1 = 262143, 只保留低 18 bit。
1516 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (2^4), 低 4 bit 恒为 0,
1517 // 可省略以扩大可表示范围。
1518 // 解码: decoded = encoded << 4 (回到原始 byte 值)。
1519 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4
1520
1521 // make_smem_desc: 构造 WGMMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
1522 // 硬件 WGMMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,
1523 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
1524 //
1525 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor):
1526 // -----
1527 // | 位域          | 范围      | 大小 | 含义          |
1528 // |-----|-----|-----|-----|
1529 // | start_address  | [0, 14)   | 14   | smem 基址编码 |
1530 // | (unused)       | [14, 16)  | 2    | -             |
1531 // | leading_byte_offset | [16, 30) | 14   | 次要维度的字节步长编码 |
1532 // | (unused)       | [30, 32)  | 2    | -             |
1533 // | stride_byte_offset | [32, 46) | 14   | 主要维度的字节步长编码 |
1534 // | (unused)       | [46, 49)  | 3    | -             |
1535 // | base_offset    | [49, 52)  | 3    | swizzle pattern 偏移 |
1536 // | (unused)       | [52, 62)  | 10   | -             |
1537 // | layout_type    | [62, 64)  | 2    | swizzle 模式   |
1538 // -----
1539 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
1540 //
1541 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2):
1542 //   - 128B swizzle atom 在 128-bit 单位下为 8×8
1543 //   - 归一化到 half(2B) 后: atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
1544 //   - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
1545 //   - 这是 stride_byte_offset=1024 的来源
1546 //
1547 //   - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1),
1548 //     此处填 16 仅为占位, 编码后为 1。
1549 //
1550 //   - base_offset=0 (bits [49,52) 未显式置位), 表示 smem ptr 已 1024B 对齐。

```

```

1551 // 若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
1552 //
1553 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
1554 __device__ inline uint64_t make_smem_desc(half *ptr) {
1555 // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
1556 // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址)。
1557 uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
1558
1559 // 从零开始: 所有位域初始化为 0。
1560 uint64_t desc = 0x0000000000000000;
1561
1562 // — bits [0, 14): start_address —
1563 // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit),
1564 // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
1565 // 可寻址范围:  $2^{(14+4)} = 2^{18} = 256K$  个 half = 512 KB 共享内存。
1566 desc |= SMEM_DESC_ENCODE(addr);
1567
1568 // — bits [16, 30): leading_byte_offset —
1569 // 原始值 16 (字节), 编码后为 (16 & 0x3FFFF) >> 4 = 1。
1570 // << 16 将编码值放到 bits [16, 30)。
1571 // K-Major + 128B swizzle 下该字段 unused (hardware assumes 1),
1572 // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
1573 // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
1574 // 对 INTERLEAVE: 同一 8x2 brick 内第一列到第二列的字节偏移。
1575 // 对 swizzle: unused, assumed to be 1。
1576 desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
1577
1578 // — bits [32, 46): stride_byte_offset —
1579 // 原始值 1024 (字节), 编码后为 (1024 & 0x3FFFF) >> 4 = 64。
1580 // << 32 将编码值放到 bits [32, 46)。
1581 // 1024 的来源 (K-Major + 128B swizzle + half):
1582 // 一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
1583 // 从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
1584 // 若为 MN-Major: SB0 是从首列到下一列的偏移 (8 列一组)。
1585 desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
1586
1587 // — bits [62, 64): layout_type (swizzle mode) —
1588 // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
1589 // 1 = 128B swizzle mode。
1590 // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
1591 // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
1592 // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
1593 desc |= 1llu << 62;
1594
1595 return desc;
1596 }
1597
1598 // ---- WGMMA PTX 指令宏 ----
1599 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
1600 // m64n128k16: M=64, N=128, K=16
1601 // f16.f16.f16: A=f16, B=f16, D=f16 (accumulation in f16)
1602 // 每线程 32 个 uint32 输出: D=64x128=8192 half, warpgroup 128 线程分担,
1603 // 每线程 64 half = 32 uint32
1604 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成)
1605 // ScaleD: 0=clear accum, 1=accumulate (用于 K 维迭代时累加)
1606 #define WGMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA, \
1607                                     TransB) \
1608 { \
1609     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
1610     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
1611     asm volatile( \
1612         "{\n" \
1613         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
1614         "{%0,  %1,  %2,  %3,  %4,  %5,  %6,  %7,  " \
1615         " %8,  %9,  %10, %11, %12, %13, %14, %15, " \
1616         " %16, %17, %18, %19, %20, %21, %22, %23, " \
1617         " %24, %25, %26, %27, %28, %29, %30, %31}, " \
1618         " %32, " \
1619         " %33, " \
1620         " %34, %35, %36, %37, %38;\n" \
1621         "}\n" \
1622         : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
1623           "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
1624           "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \

```



```

1625         "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
1626         "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
1627         "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
1628         "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
1629         "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
1630     : "l"(desc_a), "l"(desc_b), "n"(int32_t(ScaleD)), \
1631     "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
1632     "n"(int32_t(TransB))); \
1633 }
1634
1635 // ---- TMA Shared Memory Layout ----
1636 // Multi-stage pipeline: K_STAGE 个 stage, 每个 stage 存储 A[BM×BK] + B[BK×BN]
1637 //
1638 // 每个 stage 包含两块 smem:
1639 //   A tile: [BM, BK] row-major → BK×BM 个 half, 地址连续
1640 //   B tile: [BK, BN] row-major → BN×BK 个 half, 地址连续
1641 // 注意 B 的 smem 布局是 row-major [BK, BN], 物理上按 K-major 排列 (imm-trans-b=0),
1642 // 与 A 的 K-major 配合供 WGMMMA 读取。128B swizzle 将 [BK,BN] row-major 重新映射为
1643 // K-major 布局, 使得 K 维元素在 128B atom 内连续, 区别于 MMA TN 布局下的 B^T[N,K]。
1644 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
1645     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
1646     alignas(128) half B[BK * BN * QSIZE]; // B tile: row-major [BK, BN] (K-major 布局, 供 WGMMMA imm-trans-b=0 直接读取)
1647 };
1648
1649 // ---- WGMMMA Kernel: Warp Specialization + TMA ----
1650 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化):
1651 //
1652 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
1653 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMMA 支持将工作拆分为两个
1654 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
1655 //
1656 //   WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
1657 //   将 A/B tile 从 HBM 搬到 shared memory。
1658 //   WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMMA 矩阵乘。
1659 //
1660 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
1661 //   - full[qidx]: Producer 发信号表示 stage qidx 的数据已就绪
1662 //   - empty[qidx]: Consumer 发信号表示 stage qidx 的使用完毕, 可被覆盖
1663 //
1664 // 本节重点理解:
1665 //   1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
1666 //   即可搬运整个 2D tile, 无需所有线程参与。
1667 //   2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
1668 //   3) 多 stage pipeline 使 Consumer 计算 stage qidx 的同时,
1669 //   Producer 可以搬运 stage (qidx+1), 隐藏 HBM-SMEM 延迟。
1670 //
1671 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比):
1672 //   WGMMMA Atom:      m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
1673 //   K Tile:           BK=64, 每个 K tile 包含 BK/WGMMMA_K=4 个 WGMMMA atom
1674 //   M Tile:           BM=128, 每个 M tile 包含 BM/WGMMMA_M=2 个 WGMMMA atom
1675 //   N Tile:           BN=128, 单个 WGMMMA_N=128 即可覆盖, 无需在 N 方向分块
1676 //   Block Tile:       C[128,128] = A[128,64] × B[64,128]
1677 //   Threads:          256 = 2 warpgroups × 128 threads/warpgroup
1678 //   Producer(WG0):    128 threads (仅 thread 0 做 TMA 提交)
1679 //   Consumer(WG1):    128 threads = 4 warps (全部参与 WGMMMA 和写回)
1680 //
1681 // K_STAGE=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步, 确保 HBM-SMEM
1682 // 的延迟被计算完全掩盖。
1683 //
1684 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1685 //   - grid.z = S 个 swizzle 分区, 将连续 block 打散到不同 N 区域改善 L2 命中
1686 // Block: (256, 1, 1), 2 warpgroups
1687 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
1688 template <const int WGMMMA_M = 64, const int WGMMMA_N = 128,
1689         const int WGMMMA_K = 16, const int BM = 128, const int BN = 128,
1690         const int BK = 64, const int NUM_THREADS = 256, const int K_STAGE = 3,
1691         const bool BLOCK_SWIZZLE = false>
1692 __global__ void __launch_bounds__(NUM_THREADS)
1693     hgemm_wgmma_stages_tn(
1694         int M, int N, int K, half *C,
1695         const CUtensorMap *__restrict__ tensorMapA,
1696         const CUtensorMap *__restrict__ tensorMapB) {
1697
1698     // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;

```



```

1699 // 对 row-major [H,W] 矩阵, TMA shape 参数写的是 (W,H) 而不是 (H,W),
1700 // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
1701 // 不展开宿主侧 create_tensor_map 细节。
1702
1703 // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1704 // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
1705 const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1706 const int by = blockIdx.y;
1707 // num_consumers = (NUM_THREADS / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
1708 // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
1709 constexpr int num_consumers = (NUM_THREADS / 128) - 1; // 1 consumer WG
1710 // B_WG_M = BM / num_consumers: 每个 consumer warpgroup 负责的 M 行数
1711 // 当只有一个 consumer 时, 它负责全部 BM=128 行
1712 constexpr int B_WG_M = BM / num_consumers; // 128
1713
1714 // 边界检查: 确保当前 block 不超出 M/N 范围
1715 if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
1716     return;
1717
1718 // ---- Shared Memory 分配 ----
1719 // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
1720 // __align__(128) 满足 TMA 和 WGMMMA 的 16B 对齐 + 128B swizzle 对齐要求
1721 extern __shared__ __align__(128) uint8_t smem[];
1722 WgmmaSMem<BM, BN, BK, K_STAGE> &s =
1723     *reinterpret_cast<WgmmaSMem<BM, BN, BK, K_STAGE>*>(smem);
1724 half *s_a = s.A;
1725 half *s_b = s.B;
1726
1727 // ---- cuda::barrier 初始化 ----
1728 // 每个 stage 有两个 barrier:
1729 // full[qidx]: Producer thread 0 发 full 信号, 128 个 Consumer 线程等 full
1730 // empty[qidx]: Consumer 发 empty 信号, Producer thread 0 等 empty
1731 // 每轮参与 arrive 的总人数 = 128 (consumer) + 1 (producer) = 129
1732 __shared__ cuda::barrier<cuda::thread_scope_block> full[K_STAGE];
1733 __shared__ cuda::barrier<cuda::thread_scope_block> empty[K_STAGE];
1734
1735 // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。
1736 const int num_blocks_k = K / BK;
1737 const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
1738 const int tid = threadIdx.x % 128; // 0~127 within warpgroup
1739
1740 // 初始化 barriers (仅 thread 0 执行)
1741 if (threadIdx.x == 0) {
1742     for (int i = 0; i < K_STAGE; ++i) {
1743         // init 的第二个参数是 barrier 的 arrive count:
1744         // num_consumers * 128 + 1 = 1 * 128 + 1 = 129
1745         // 即每轮需要 128 个 consumer 线程 + 1 个 producer 线程都 arrive 后,
1746         // barrier 才翻转 phase 并唤醒等待线程。
1747         init(&full[i], num_consumers * 128 + 1); // 128 consumer + 1 producer
1748         init(&empty[i], num_consumers * 128 + 1); // same
1749     }
1750     // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
1751     // 对 async proxy (TMA/WGMMMA) 可见。
1752     cuda::device::fence_proxy_async_shared_cta();
1753 }
1754 __syncthreads();
1755
1756 // =====
1757 // Producer Warpgroup (WG0, threadIdx.x 0~127)
1758 // 职责: 提交 TMA 2D 拷贝, 将 A/B tile 从 HBM 异步搬运到 SMEM。
1759 // 只有 tid==0 执行实际拷贝提交, 其余 127 个线程空闲。
1760 // =====
1761 if (wg_idx == 0) {
1762     if (tid == 0) {
1763         // qidx: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
1764         int qidx = 0;
1765         for (int block_k_iter = 0; block_k_iter < num_blocks_k;
1766              ++block_k_iter, ++qidx) {
1767             if (qidx == K_STAGE)
1768                 qidx = 0;
1769
1770             // Step P1: 等待 Consumer 释放此 stage (empty 信号)
1771             // empty[qidx].arrive() 表示 Producer 自己"已准备好等待",
1772             // 然后 wait() 阻塞直到 barrier phase 翻转 (即所有 Consumer 都已 arrive)。

```

```

1773     empty[qidx].wait(empty[qidx].arrive());
1774
1775     // Step P2: 提交 TMA 2D 拷贝指令
1776     // cp_async_bulk_tensor_2d_global_to_shared:
1777     // 参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
1778     // 参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
1779     // 参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
1780     // 参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
1781     //
1782     // TMA 是硬件 DMA 引擎: 一次指令提交即可搬运整个 2D tile,
1783     // 无需线程逐元素搬运, 零寄存器开销。
1784     // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
1785     cuda::device::cp_async_bulk_tensor_2d_global_to_shared(
1786         &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
1787         full[qidx]);
1788
1789     // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
1790     cuda::device::cp_async_bulk_tensor_2d_global_to_shared(
1791         &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
1792         full[qidx]);
1793
1794     // Step P3: 通知 Consumer 此 stage 已准备好 (full 信号)
1795     // barrier_arrive_tx 除了 arrive 之外还额外告知硬件:
1796     // 本次 TMA 事务预期传输的字节数。
1797     // Consumer 的 full[qidx].wait() 会等待这 (BK*BN+BK*BM)*sizeof(half)
1798     // 字节全部写入 smem 后才返回。
1799     cuda::device::barrier_arrive_tx(
1800         full[qidx], 1, (BK * BN + BK * BM) * sizeof(half));
1801 }
1802 }
1803 }
1804 // =====
1805 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
1806 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
1807 // 所有 128 个线程 (4 warps) 全部参与。
1808 // =====
1809 else {
1810     // Step C0: Consumer 初始时"准备就绪"
1811     // 对所有 stage 的 empty barrier 执行 arrive, 表示初始时所有 stage
1812     // 都是"空的" (可被 Producer 写入)。
1813     // 如果没有这一步, Producer 的 empty[qidx].wait() 在第一轮会永远阻塞。
1814     for (int i = 0; i < K_STAGE; ++i) {
1815         empty[i].arrive();
1816     }
1817
1818     // 累加器寄存器声明
1819     // d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4]:
1820     // - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
1821     // - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
1822     // - d[*][g][*]: N 方向第 g 组 16 列
1823     // - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16x16 子块)
1824     // 每个线程总共 2 * 8 * 4 = 64 uint32 = 128 half
1825     // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
1826     uint32_t d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4] = {};
1827
1828     int qidx = 0;
1829     // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
1830     for (int block_k_iter = 0; block_k_iter < num_blocks_k;
1831          ++block_k_iter, ++qidx) {
1832         if (qidx == K_STAGE)
1833             qidx = 0;
1834
1835         // Step C1: 等待 Producer 的 full 信号
1836         // 表示 stage qidx 的 A/B tile 数据已通过 TMA 搬运到 smem。
1837         full[qidx].wait(full[qidx].arrive());
1838
1839         // Step C2: 发射 WGMMMA 指令序列
1840         //
1841         // WGMMMA 指令序列的固定模式:
1842         // WGMMMA_FENCE -> 发射一串 WGMMMA -> COMMIT_GROUP -> WAIT_GROUP
1843         //
1844         // WGMMMA_FENCE: 确保 smem 写可见、accum 寄存器准备好。
1845         // 是一条内存 fence 指令, 建立 async proxy 与 generic proxy 之间的
1846         // 可见性顺序。必须在发射 WGMMMA 之前执行。

```

```

1847 //
1848 // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
1849 // 执行 D[64*128] += A[64*16] * B[16*128]
1850 // 其中 A 的 smem 指针由 descA 描述 (K-major), B 由 descB 描述 (K-major)
1851 // imm-trans-a=0/imm-trans-b=0 表示 A/B 都是 K-major (非转置)。
1852 // ScaleD=1: 累加模式 (不清零), 用于 K 维累加。
1853 // ScaleA=ScaleB=1: 不翻转符号。
1854 WGMMMA_FENCE();
1855
1856 // M 维迭代: BM/WGMMMA_M = 128/64 = 2 个 WGMMMA atom
1857 // 每个 atom 处理 64 行 M 方向数据。
1858 #pragma unroll
1859 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
1860 // wgmma_sA 指向当前 stage 中 A tile 的第 m_it 个 64*BK 子块
1861 // s_a 布局: [K_STAGE][BM][BK] -> qidx * BK*BM + BK * (m_it*64)
1862 half *wgmma_sA = s_a + qidx * BK * BM + BK * m_it * WGMMMA_M;
1863
1864 // K 维迭代: BK/WGMMMA_K = 64/16 = 4 次 WGMMMA (累加)
1865 // 每次处理 K=16 维的矩阵乘, 4 次累加后覆盖完整的 BK=64。
1866 #pragma unroll
1867 for (int k_it = 0; k_it < BK / WGMMMA_K; ++k_it) {
1868 // 第 k_it 次 WGMMMA:
1869 // A: wgmma_sA + k_it * WGMMMA_K (A 的 K 维起始位置)
1870 // B: s_b + qidx * BK * BN + k_it * WGMMMA_K (B 的 K 维起始位置)
1871 // 注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
1872 // 第 k_it*16 行开始取 16 行, 每行 BN=128 列。
1873 // ScaleD=1: 累加 (K 维迭代需要累积结果)
1874 WGMMMA_M64N128K16_F16F16F16(d[m_it], wgmma_sA + k_it * WGMMMA_K,
1875 s_b + qidx * BK * BN + k_it * WGMMMA_K,
1876 1, // ScaleD=1: accumulate (不清零)
1877 1, 1, 0, 0);
1878 }
1879 }
1880
1881 // Step C3: 提交并等待 WGMMMA 完成
1882 // COMMIT_GROUP: 将此前发射的所有 WGMMMA 指令归为一组提交。
1883 // WAIT_GROUP(0): 等待之前 commit 的所有 WGMMMA 完成 (0 表示等待所有组)。
1884 // 注意 WGMMMA 是异步执行 (async proxy), 这些指令用于同步 completion。
1885 WGMMMA_COMMIT_GROUP();
1886 WGMMMA_WAIT_GROUP(0);
1887
1888 // Step C4: 释放此 stage (发 empty 信号给 Producer)
1889 // 通知 Producer stage qidx 的 smem 已使用完毕, 可以安全覆盖。
1890 empty[qidx].arrive();
1891 }
1892
1893 // =====
1894 // Epilogue: 将寄存器中的累加结果写回 global memory C
1895 // =====
1896 //
1897 // WGMMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
1898 //
1899 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
1900 // 这些 half 分布在 d[2][8][4] 数组中:
1901 // d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
1902 // d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
1903 // d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
1904 // d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
1905 //
1906 // 其中:
1907 // row = warp * 16 + lane / 4 (0~63, 4 warps * 16 rows)
1908 // col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
1909 //
1910 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
1911 // +-----+-----+
1912 // | reg[0] | reg[2] | <- rows [row, row+8)
1913 // |(col,+2)|(col+8)|
1914 // +-----+-----+
1915 // | reg[1] | reg[3] | <- rows [row+8, row+16)
1916 // |(col,+2)|(col+8)|
1917 // +-----+-----+
1918 // 每个 reg 包含 2 个连续 half (col 和 col+2), 用 uint32 存储。
1919 //
1920 // 三维遍历:

```

```

1921 // m_it=0: rows 0~63, m_it=1: rows 64~127
1922 // g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
1923 // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half.
1924 // 128 线程 * 128 half = 16384 half = 128 * 128 ok
1925
1926 const int lane = tid % 32;
1927 const int warp = tid / 32;
1928 // row: 当前线程在当前 WGMMMA atom 中负责的起始行
1929 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]
1930 const int row = warp * 16 + lane / 4;
1931 // block_C: 当前 C tile 在 global memory 中的起始地址
1932 // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
1933 half *block_C = C + by * BM * N + bx * BN;
1934
1935 #pragma unroll
1936 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
1937     int yo = m_it * WGMMMA_M; // M 方向行偏移 (0 或 64)
1938     #pragma unroll
1939     for (int g = 0; g < WGMMMA_N / 16; ++g) {
1940         int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
1941         // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
1942         // 左上象限: (row, col)
1943         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) =
1944             d[m_it][g][0];
1945         // 左下象限: (row+8, col)
1946         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) =
1947             d[m_it][g][1];
1948         // 右上象限: (row, col+8)
1949         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) =
1950             d[m_it][g][2];
1951         // 右下象限: (row+8, col+8)
1952         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) =
1953             d[m_it][g][3];
1954     }
1955 }
1956 }
1957 }
1958 #endif /* __CUDA_ARCH__ >= 900 */
1959
1960 // =====
1961 // Phase 8: FlashAttention-2 (Split-Q + MMA m16n8k16)
1962 // =====
1963 // 面试要点 (FlashAttention 算法):
1964 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
1965 //    但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
1966 // 2. FA 三板斧:
1967 //    a) Tiling: Q 分块 [Br,d], K/V 沿 seqlen 分块 [Bc,d]
1968 //    b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
1969 //    c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
1970 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
1971 //    - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
1972 //    - 优点: 减少 warp 间通信和 shuffle
1973 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691):
1974 //    for each K,V tile:
1975 //        S_cur = Q @ K^T // 未缩放, 存入 R_S
1976 //        m_new = max(m_old, row_max(S_cur * scale))
1977 //        P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
1978 //        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
1979 //        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
1980 //        O_final = O_new / l_final
1981 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
1982 //    - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
1983 //    - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
1984 //      后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
1985 //    - 这是此实现的寄存器布局复用技巧, 不要背成“所有 MMA A/C fragment
1986 //      都天然同构”的通用结论
1987 //
1988 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
1989 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
1990 // Grid: ((QKV_seqlen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
1991 // Block: (128, 1, 1), kNumThreads=WARP_SIZE*kMmaTileSeqLenQ*kMmaTileSeqLenK=128
1992 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
1993
1994 // ---- 寄存器填充辅助函数 ----

```

```

1995 template <typename T, int M, const int N, const int K = 2>
1996 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
1997     #pragma unroll
1998     for (int i = 0; i < M; ++i)
1999     #pragma unroll
2000         for (int j = 0; j < N; ++j)
2001     #pragma unroll
2002         for (int k = 0; k < K; ++k)
2003             R[i][j][k] = val;
2004 }
2005
2006 template <typename T, int M, const int N = 2>
2007 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
2008     #pragma unroll
2009     for (int i = 0; i < M; ++i)
2010     #pragma unroll
2011         for (int j = 0; j < N; ++j)
2012             R[i][j] = val;
2013 }
2014
2015 // =====
2016 // FlashAttention-2 Split-Q Kernel (完整实现)
2017 // =====
2018 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
2019 //
2020 // Tile 设计 (以 kHeadDim=64 为例) :
2021 //   Br = kMmaAtomM * kMmaTileSeqLenQ * kWarpTileSeqLenQ = 16*4*1 = 64
2022 //   Bc = kMmaAtomN * kMmaTileSeqLenK * kWarpTileSeqLenK = 8*1*8 = 64
2023 //   Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
2024 //
2025 // 执行流程:
2026 //   1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
2027 //   2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
2028 //   3)   3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
2029 //   4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMM16816
2030 //   5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
2031 //       → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
2032 //   6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
2033 //       → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
2034 //   7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
2035 //   8) 最终 rescale: O_final = (1/L_final) * O_final
2036 //   9) Epilogue: warp shuffle + 128-bit collective store
2037
2038 template <
2039     const int kHeadDim,           // head dim: 32, 64, 128
2040     const int kMmaAtomM,          // 16 (MMA instruction M dimension)
2041     const int kMmaAtomN,          // 8 (MMA instruction N dimension)
2042     const int kMmaAtomK,          // 16 (MMA instruction K dimension)
2043     const int kMmaTileSeqLenQ,    // MMA tiles along Q's M dim, 4 → Br=16*4=64
2044     const int kMmaTileSeqLenK,    // MMA tiles along K's N dim, 1 → Bc basis=8
2045     const int kMmaTileSeqLenP,    // MMA tiles for P@V M dim, must equal
2046                                     // kMmaTileSeqLenQ
2047     const int kMmaTileHeadDimV,    // MMA tiles for P@V N dim (head dim direction)
2048     const int kWarpTileSeqLenQ,    // warp tiles along Q's M, 1 → Br per warp=16
2049     const int kWarpTileSeqLenK,    // warp tiles along K's N, 8 → Bc_warp=8*8=64
2050     const int kWarpTileSeqLenP,    // warp tiles for P@V M dim, 1
2051     const int kWarpTileHeadDimV,   // warp tiles for P@V N dim,
2052                                     // kHeadDim/(8*kMmaTileHeadDimV)
2053     const int kStage,              // pipeline stages for K: 1 or 2
2054     const int kPad>,              // padding for bank conflict avoidance
2055     __global__ void __launch_bounds__(WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK)
2056     flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,
2057                                     int QKV_seqlen, int QKV_head) {
2058
2059     // Tile dimensions
2060     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kWarpTileSeqLenQ; // 64
2061     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kWarpTileSeqLenK; // 64
2062     constexpr int kNumThreads =
2063         WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
2064     const int Tc = (QKV_seqlen + Bc - 1) / Bc;
2065     // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
2066     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
2067     const float scale = 1.0f / sqrtf((float)kHeadDim);
2068

```

```

2069 // Block indexing
2070 const int QKV_batch_id = blockIdx.y / QKV_head;
2071 const int QKV_head_id = blockIdx.y % QKV_head;
2072 const int Q_tile_id = blockIdx.x;
2073 const int tid = threadIdx.x;
2074 const int warp_id = tid / WARP_SIZE;
2075 const int lane_id = tid % WARP_SIZE;
2076 const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
2077 const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
2078
2079 // Global memory base offsets for this (batch, head)
2080 // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
2081 const int Q_gmem_offset =
2082     (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
2083     kHeadDim;
2084 const int K_gmem_offset = Q_gmem_offset;
2085 const int V_gmem_offset = Q_gmem_offset;
2086 const int O_gmem_offset = Q_gmem_offset;
2087
2088 // Thread-to-smem mapping for cooperative load
2089 int load_smem_Q_Br = tid / (kNumThreads / Br);
2090 int load_smem_Q_d =
2091     (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
2092 int load_smem_K_Bc = tid / (kNumThreads / Bc);
2093 int load_smem_K_d =
2094     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
2095 int load_smem_V_Bc = tid / (kNumThreads / Bc);
2096 int load_smem_V_d =
2097     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
2098
2099 int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
2100 if (load_gmem_Q_Br >= QKV_seqlen)
2101     return;
2102
2103 // ---- Shared memory layout ----
2104 extern __shared__ half smem[];
2105 constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+kPad]
2106 constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
2107 half *Q_tile_smem = smem;
2108 half *K_tile_smem = Q_tile_smem + Q_tile_size;
2109 half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
2110 // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
2111 // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
2112
2113 uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
2114 uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
2115 uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
2116
2117 // ---- Online Softmax persistent state ----
2118 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
2119 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
2120 float lane_block_row_max_old[kWarpTileSeqLenQ][2];
2121 float lane_block_row_sum_old[kWarpTileSeqLenQ][2];
2122 fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
2123 fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
2124
2125 // ---- Register allocation ----
2126 uint32_t R_Q[kWarpTileSeqLenQ][4]; // Q regs
2127 uint32_t R_K[kWarpTileSeqLenK][2]; // K regs
2128 uint32_t R_V[kWarpTileHeadDimV][2]; // V regs
2129 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
2130 // 对单个 m16n8k16 tile 而言:
2131 // - reg[0] 持有该 tile 前 8 行里的两个 half 值
2132 // - reg[1] 持有该 tile 后 8 行里的两个 half 值
2133 // 后续 softmax, P@V, online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
2134 uint32_t R_S[kWarpTileSeqLenQ][kWarpTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
2135 uint32_t R_O[kWarpTileSeqLenP][kWarpTileHeadDimV][2]; // O for current tile
2136 uint32_t R_D[kWarpTileSeqLenP][kWarpTileHeadDimV][2]; // O accumulator (final output)
2137
2138
2139 fill_3D_regs<uint32_t, kWarpTileSeqLenQ, kWarpTileSeqLenK, 2>(R_S, 0);
2140 fill_3D_regs<uint32_t, kWarpTileSeqLenP, kWarpTileHeadDimV, 2>(R_D, 0);
2141
2142 // =====

```



```

2143 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
2144 // =====
2145 {
2146     int load_gmem_Q_addr =
2147         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
2148     uint32_t load_smem_Q_ptr =
2149         smem_Q_base_ptr +
2150         (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
2151 #pragma unroll
2152     for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
2153         CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
2154     }
2155     CP_ASYNC_COMMIT_GROUP();
2156 }
2157
2158 // =====
2159 // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
2160 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqlen 遍历始终从 tile 0 开始,
2161 // 后续在外循环里不断递增到 tile 1/2/3/.../Tc-1。
2162 // =====
2163 if constexpr (kStage > 1) {
2164 #pragma unroll
2165     for (int stage = 0; stage < (kStage - 1); ++stage) {
2166         int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
2167         int load_gmem_K_addr =
2168             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
2169         uint32_t load_smem_K_ptr =
2170             smem_K_base_ptr +
2171             (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
2172              load_smem_K_d) *
2173             sizeof(half);
2174 #pragma unroll
2175         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2176             CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
2177         }
2178         CP_ASYNC_COMMIT_GROUP();
2179     }
2180     CP_ASYNC_WAIT_GROUP(kStage - 2);
2181     __syncthreads();
2182 }
2183
2184 // =====
2185 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
2186 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
2187 // =====
2188 #pragma unroll 1
2189 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
2190     int smem_sel = tile_K_seqlen % kStage;
2191     int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;
2192
2193     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
2194     if constexpr (kStage > 1) {
2195         // 只有 kStage>1 才能真正做 K 的 pipeline:
2196         // smem_sel 负责 “当前正在计算” 的 K tile, smem_sel_next 负责 “下一轮预取” 的 K tile。
2197         // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。
2198         // Load current V tile (no pipeline for V — one stage is enough)
2199         {
2200             int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
2201             int load_gmem_V_addr =
2202                 V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
2203             uint32_t load_smem_V_ptr =
2204                 smem_V_base_ptr +
2205                 (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
2206 #pragma unroll
2207             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2208                 CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
2209             }
2210             CP_ASYNC_COMMIT_GROUP();
2211         }
2212
2213         // Prefetch next K tile (pipelined)
2214         if ((tile_K_seqlen + 1) < Tc) {
2215             int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
2216             int load_gmem_K_addr =

```



```

2217         K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
2218     uint32_t load_smem_K_ptr =
2219         smem_K_base_ptr +
2220         (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
2221          load_smem_K_d) *
2222         sizeof(half);
2223 #pragma unroll
2224     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2225         CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
2226     }
2227     CP_ASYNC_COMMIT_GROUP();
2228 }
2229 }
2230
2231 // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
2232 fill_3D_regs<uint32_t, kWarpTileSeqLenQ, kWarpTileSeqLenK, 2>(R_S, 0);
2233 #pragma unroll
2234 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
2235     // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
2236 #pragma unroll
2237     for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
2238         int warp_smem_Q_Br =
2239             warp_QP * (kMmaAtomM * kWarpTileSeqLenQ) + i * kMmaAtomM;
2240         int lane_smem_Q_Br =
2241             warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
2242         int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
2243         uint32_t lane_smem_Q_ptr =
2244             smem_Q_base_ptr +
2245             (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
2246         LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
2247                    lane_smem_Q_ptr);
2248     }
2249
2250     // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
2251     // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
2252 #pragma unroll
2253     for (int j = 0; j < kWarpTileSeqLenK; ++j) {
2254         int warp_smem_K_Bc =
2255             warp_KV * (kMmaAtomN * kWarpTileSeqLenK) + j * kMmaAtomN;
2256         int lane_smem_K_Bc =
2257             warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
2258         int lane_smem_K_d =
2259             tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
2260         uint32_t lane_smem_K_ptr =
2261             smem_K_base_ptr +
2262             (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
2263              lane_smem_K_d) *
2264             sizeof(half);
2265         LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
2266     }
2267
2268     // MMA: S[tile] += Q[tile] @ K^T[tile]
2269 #pragma unroll
2270     for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
2271 #pragma unroll
2272         for (int j = 0; j < kWarpTileSeqLenK; ++j) {
2273             HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
2274                      R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
2275                      R_S[i][j][1]);
2276         }
2277     }
2278 } // end loop over d
2279 __syncthreads();
2280
2281 // =====
2282 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to
2283 // R_S
2284 // =====
2285 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
2286 // - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
2287 // - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
2288 // - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
2289 // - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
2290 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是

```

```

2291 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
2292 // Each (i, j) pair = one 16x8 MMA tile; there are kWarpTileSeqLenQ x
2293 // kWarpTileSeqLenK tiles.
2294
2295 float lane_row_max_new[kWarpTileSeqLenQ][2];
2296 float lane_row_sum_new[kWarpTileSeqLenQ][2];
2297 fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
2298 fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
2299
2300 // ---- Pass 1: Thread-level reduce max across all columns (kWarpTileSeqLenK
2301 // tiles) ----
2302 #pragma unroll
2303 for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
2304 #pragma unroll
2305 for (int j = 0; j < kWarpTileSeqLenK; ++j) {
2306 // Extract half values from R_S registers (C matrix fragment layout)
2307 float2 t_reg_S_0 =
2308     __half22float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
2309 float2 t_reg_S_1 =
2310     __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
2311 // S = (Q@K^T) * scale
2312 float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
2313 float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
2314 lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
2315 lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
2316 }
2317 // Warp-level reduce max (warp_size = 4 for Q@K^T — only lanes
2318 // {0,4,8,...,28} hold valid data)
2319 lane_row_max_new[i][0] =
2320     warp_reduce_max<4, float>(lane_row_max_new[i][0]);
2321 lane_row_max_new[i][1] =
2322     warp_reduce_max<4, float>(lane_row_max_new[i][1]);
2323 }
2324
2325 // ---- Pass 2: Compute P = exp(S*scale - m_new), store back to R_S ----
2326 // 面试关键点: 这里将 P 写回 R_S 寄存器!
2327 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
2328 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
2329 #pragma unroll
2330 for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
2331 // m_new = max(m_old, m_cur)
2332 float block_row_max_new_0 =
2333     max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
2334 float block_row_max_new_1 =
2335     max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
2336
2337 #pragma unroll
2338 for (int j = 0; j < kWarpTileSeqLenK; ++j) {
2339 float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
2340 float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
2341
2342 // P = exp(S * scale - m_new), 用 fma 保证精度
2343 t_reg_S_0.x =
2344     __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
2345 t_reg_S_0.y =
2346     __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
2347 t_reg_S_1.x =
2348     __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
2349 t_reg_S_1.y =
2350     __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
2351
2352 // Accumulate row sums
2353 lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
2354 lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
2355
2356 // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
2357 HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
2358 HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
2359 }
2360
2361 // Warp-level reduce sum (warp_size = 4, same as max)
2362 lane_row_sum_new[i][0] =
2363     warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
2364 lane_row_sum_new[i][1] =

```

```

2365     warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
2366 }
2367 __syncthreads();
2368
2369 // =====
2370 // 3d: P@V — P[Br,Bc] @ V[Bc,d] = O[Br,d]
2371 // =====
2372 // Wait for V to be ready before computing P@V
2373 if constexpr (kStage > 1) {
2374     if ((tile_K_seqlen + 1) < Tc) {
2375         CP_ASYNC_WAIT_GROUP(1);
2376     } else {
2377         CP_ASYNC_WAIT_GROUP(0);
2378     }
2379 } else {
2380     CP_ASYNC_WAIT_GROUP(0);
2381 }
2382 __syncthreads();
2383
2384 fill_3D_regs<uint32_t, kWarpTileSeqLenP, kWarpTileHeadDimV, 2>(R_0, 0);
2385
2386 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
2387 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
2388 #pragma unroll
2389 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
2390     // ldmatrix.x2.trans: load V[Bc,d] with transposition
2391     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
2392     // transposed ldmatrix
2393 #pragma unroll
2394 for (int j = 0; j < kWarpTileHeadDimV; ++j) {
2395     int warp_smem_V_d =
2396         warp_KV * (kMmaAtomN * kWarpTileHeadDimV) + j * kMmaAtomN;
2397     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
2398     int lane_smem_V_d = warp_smem_V_d;
2399     uint32_t lane_smem_V_ptr =
2400         smem_V_base_ptr +
2401         (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);
2402     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
2403 }
2404
2405 // P matrix layout in R_S[i][j][2]:
2406 //   MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
2407 //   | 64x64 |      warp_KV 0      |
2408 //   | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
2409 //   | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
2410 //   | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
2411 //   | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
2412 // tile_V_Bc selects which 16-column slice of P to use:
2413 //   tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
2414 //   tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
2415 //   tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
2416 //   tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
2417 // 对应的 MMA A fragment 布局可以这样记:
2418 //   - rows 0~7: lane 0~3 → {a0,a1} 与 {a4,a5}, lane 4~7 → 下一行, 依次类推
2419 //   - rows 8~15: lane 0~3 → {a2,a3} 与 {a6,a7}, lane 4~7 → 下一行, 依次类推
2420 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
2421 // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
2422 // fragment 都无条件成立的通用结论。
2423 // layout转换逻辑:
2424 //   C layout: 8 x (16 x 8) layout → A layout: 4 x (16 x 16) layout
2425 int w = tile_V_Bc * 2;
2426 #pragma unroll
2427 for (int i = 0; i < kWarpTileSeqLenP; ++i) {
2428 #pragma unroll
2429 for (int j = 0; j < kWarpTileHeadDimV; ++j) {
2430     HMMA16816(R_0[i][j][0], R_0[i][j][1], // C fragment output
2431              R_S[i][w][0], R_S[i][w][1], R_S[i][w+1][0], R_S[i][w+1][1], // A fragment = P
2432              R_V[j][0], R_V[j][1], // B fragment = V
2433              R_0[i][j][0], R_0[i][j][1]); // C fragment output
2434     }
2435 }
2436 } // end for tile_V_Bc
2437 __syncthreads();
2438

```

```

2439 // =====
2440 // 3e: Online rescaling —  $O_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}}) * O_{\text{old}} + P@V$ 
2441 // =====
2442 // 公式来源: FA2 paper Eq.(7-8), 使用  $\exp(m_{\text{old}} - m_{\text{new}})$  做  $O$  与  $l$  的 rescale
2443 #pragma unroll
2444 for (int i = 0; i < kWarpTileSeqLenP; ++i) {
2445     float block_row_max_new_0 = lane_row_max_new[i][0];
2446     float block_row_max_new_1 = lane_row_max_new[i][1];
2447     float block_row_sum_new_0 = lane_row_sum_new[i][0];
2448     float block_row_sum_new_1 = lane_row_sum_new[i][1];
2449
2450     float block_row_max_old_0 = lane_block_row_max_old[i][0];
2451     float block_row_max_old_1 = lane_block_row_max_old[i][1];
2452
2453     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
2454     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
2455
2456     // Handle first iteration:  $m_{\text{old}} = -\text{inf}$ , need to use  $m_{\text{new}}$  directly
2457     block_row_max_old_0 =
2458         (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
2459     block_row_max_old_1 =
2460         (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
2461
2462     float rescale_o_factor_0 =
2463         __expf(block_row_max_old_0 - block_row_max_new_0);
2464     float rescale_o_factor_1 =
2465         __expf(block_row_max_old_1 - block_row_max_new_1);
2466
2467     // Rescale  $O_{\text{old}}$  + Add  $P@V$  in one fused step
2468 #pragma unroll
2469     for (int j = 0; j < kWarpTileHeadDimV; ++j) {
2470         //  $R_0 / R_D$  与前面的  $R_S$  一样, 都按 MMA C fragment 布局解释:
2471         // reg[0] -> rows 0-7 的 {c0,c1}
2472         // reg[1] -> rows 8-15 的 {c2,c3}
2473         float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
2474         float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
2475         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
2476         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
2477
2478         //  $O_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}}) * O_{\text{old}} + P@V$  (fused multiply-add)
2479         t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
2480         t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
2481         t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
2482         t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
2483
2484         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
2485         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
2486     }
2487
2488     // Update  $l$ :  $l_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}}) * l_{\text{old}} + \text{row\_sum}(P)$ 
2489     float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
2490     float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
2491     lane_block_row_sum_old[i][0] = __fmaf_rn(
2492         rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
2493     lane_block_row_sum_old[i][1] = __fmaf_rn(
2494         rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
2495
2496     // Update  $m$ 
2497     lane_block_row_max_old[i][0] = block_row_max_new_0;
2498     lane_block_row_max_old[i][1] = block_row_max_new_1;
2499 }
2500
2501 // Wait for next K tile to be ready in smem before next iteration
2502 if constexpr (kStage > 1) {
2503     if ((tile_K_seqlen + 1) < Tc) {
2504         CP_ASYNC_WAIT_GROUP(0);
2505     }
2506     __syncthreads();
2507 }
2508 } // end outer loop over K seqlen
2509 __syncthreads();
2510
2511 // =====
2512 // Step 4: 最终 rescale —  $O_{\text{final}} = (1/l_{\text{final}}) * O_{\text{final}}$ 

```

```

2513 // =====
2514 #pragma unroll
2515 for (int i = 0; i < kWarpTileSeqLenP; ++i) {
2516     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
2517     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
2518 #pragma unroll
2519     for (int j = 0; j < kWarpTileHeadDimV; ++j) {
2520         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
2521         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
2522         t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
2523         t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
2524         t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
2525         t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
2526         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
2527         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
2528     }
2529 }
2530
2531 // =====
2532 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
2533 // =====
2534 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
2535 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
2536 #pragma unroll
2537 for (int i = 0; i < kWarpTileSeqLenP; ++i) {
2538 #pragma unroll
2539     for (int j = 0; j < kWarpTileHeadDimV; ++j) {
2540         uint32_t R_Z[2][4];
2541         R_Z[0][0] = R_D[i][j][0];
2542         R_Z[1][0] = R_D[i][j][1];
2543         R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
2544         R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
2545         R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
2546         R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
2547         R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
2548         R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
2549
2550         // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
2551         if (lane_id % 4 == 0) {
2552             int store_warp_regs_0_Br =
2553                 warp_QP * (kMmaAtomM * kWarpTileSeqLenP) + i * kMmaAtomM;
2554             int store_lane_gmem_0_Br =
2555                 Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
2556             int store_warp_regs_0_d =
2557                 warp_KV * (kMmaAtomN * kWarpTileHeadDimV) + j * kMmaAtomN;
2558             int store_lane_gmem_0_d = store_warp_regs_0_d;
2559             int store_gmem_0_addr_0 = 0_gmem_offset +
2560                 (store_lane_gmem_0_Br + 0) * kHeadDim +
2561                 store_lane_gmem_0_d;
2562             int store_gmem_0_addr_1 = 0_gmem_offset +
2563                 (store_lane_gmem_0_Br + 8) * kHeadDim +
2564                 store_lane_gmem_0_d;
2565             // LDST128BITS = reinterpret_cast<float4*>
2566             *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
2567                 *reinterpret_cast<float4*>(&R_Z[0][0]);
2568             *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
2569                 *reinterpret_cast<float4*>(&R_Z[1][0]);
2570         }
2571     }
2572 }
2573 }
2574
2575 // =====
2576 // 以下是测试代码, 验证 Phase 1 - Phase 8 的kernel的正确性, 不评估性能。
2577 // =====
2578
2579 static inline void check(cudaError_t err, const char *msg) {
2580     if (err != cudaSuccess) {
2581         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
2582         exit(EXIT_FAILURE);
2583     }
2584 }
2585
2586 }

```

```

2587 static void test_block_reduce(int N) {
2588
2589     srand(42);
2590     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2591     for (int i = 0; i < N; i++)
2592         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2593
2594     // CPU reference: sum of all elements
2595     double ref = 0.0;
2596     for (int i = 0; i < N; i++) ref += (double)h_a[i];
2597
2598     float *d_a, *d_y;
2599     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
2600     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
2601
2602     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
2603     check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
2604
2605     dim3 block(128);
2606     dim3 grid((N + 127) / 128);
2607     block_reduce_v2<<<grid, block>>>(d_a, d_y, N);
2608     check(cudaGetLastError(), "blockreduce launch");
2609     check(cudaDeviceSynchronize(), "blockreduce sync");
2610
2611     float result;
2612     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
2613
2614     float err = fabsf(result - (float)ref);
2615     printf("| %-35s | %.6e | %-4s |\n", "BlockReduceAll", err,
2616           err < 1e-2f ? "PASS" : "FAIL");
2617
2618     free(h_a);
2619     cudaFree(d_a); cudaFree(d_y);
2620 }
2621
2622 static void test_dot(int N) {
2623
2624     srand(42);
2625     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2626     float *h_b = (float *)malloc((size_t)N * sizeof(float));
2627     for (int i = 0; i < N; i++) {
2628         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2629         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2630     }
2631
2632     // CPU reference
2633     double ref = 0.0;
2634     for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
2635
2636     float *d_a, *d_b, *d_y;
2637     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
2638     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
2639     check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
2640
2641     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
2642     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
2643     check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
2644
2645     dim3 block(128);
2646     dim3 grid((N + 127) / 128);
2647     dot<<<grid, block>>>(d_a, d_b, d_y, N);
2648     check(cudaGetLastError(), "dot launch");
2649     check(cudaDeviceSynchronize(), "dot sync");
2650
2651     float result;
2652     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
2653
2654     float err = fabsf(result - (float)ref);
2655     printf("| %-35s | %.6e | %-4s |\n", "Dot", err,
2656           err < 1e-2f ? "PASS" : "FAIL");
2657
2658     // ---- Dot Vec4 ----
2659     check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
2660

```

```

2661 dim3 block_v4(32);
2662 dot_vec4<<<grid, block_v4>>>(d_a, d_b, d_y, N);
2663 check(cudaGetLastError(), "dot_vec4 launch");
2664 check(cudaDeviceSynchronize(), "dot_vec4 sync");
2665
2666 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
2667 float err_v4 = fabsf(result - (float)ref);
2668 printf("| %-35s | %.6e | %-4s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL");
2669
2670 free(h_a); free(h_b);
2671 cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
2672 }
2673
2674
2675 static void test_relu(int N) {
2676     srand(42);
2677     float *h_x = (float *)malloc((size_t)N * sizeof(float));
2678     float *h_y = (float *)malloc((size_t)N * sizeof(float));
2679     for (int i = 0; i < N; i++)
2680         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2681
2682     float *d_x, *d_y;
2683     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
2684     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
2685     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
2686
2687     for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
2688     dim3 block256(256);
2689     dim3 grid256((N + 255) / 256);
2690     relu<<<grid256, block256>>>(d_x, d_y, N);
2691     check(cudaGetLastError(), "relu launch");
2692     check(cudaDeviceSynchronize(), "relu sync");
2693     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
2694     float max_err = 0.0f;
2695     for (int i = 0; i < N; i++) {
2696         float expected = fmaxf(0.0f, h_x[i]);
2697         float err = fabsf(h_y[i] - expected);
2698         if (err > max_err) max_err = err;
2699     }
2700     printf("| %-35s | %.6e | %-4s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2701
2702     dim3 block64(64);
2703     relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
2704     check(cudaGetLastError(), "relu_vec4 launch");
2705     check(cudaDeviceSynchronize(), "relu_vec4 sync");
2706     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
2707     max_err = 0.0f;
2708     for (int i = 0; i < N; i++) {
2709         float expected = fmaxf(0.0f, h_x[i]);
2710         float err = fabsf(h_y[i] - expected);
2711         if (err > max_err) max_err = err;
2712     }
2713     printf("| %-35s | %.6e | %-4s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2714
2715     free(h_x); free(h_y);
2716     cudaFree(d_x); cudaFree(d_y);
2717 }
2718
2719
2720 static void test_elementwise(int N) {
2721     srand(42);
2722     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2723     float *h_b = (float *)malloc((size_t)N * sizeof(float));
2724     for (int i = 0; i < N; i++) {
2725         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2726         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2727     }
2728
2729     float *d_a, *d_b, *d_c;
2730     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
2731     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
2732     check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
2733     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
2734     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");

```



```

2735 dim3 block256(256);
2736 dim3 grid256((N + 255) / 256);
2737 elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
2738 check(cudaGetLastError(), "eadd launch");
2739 check(cudaDeviceSynchronize(), "eadd sync");
2740 float *h_c = (float *)malloc((size_t)N * sizeof(float));
2741 check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
2742 float max_err = 0.0f;
2743 for (int i = 0; i < N; i++) {
2744     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
2745     if (err > max_err) max_err = err;
2746 }
2747 printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2748
2749 dim3 block64(64);
2750 check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
2751 elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
2752 check(cudaGetLastError(), "eadd_vec4 launch");
2753 check(cudaDeviceSynchronize(), "eadd_vec4 sync");
2754 check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
2755 max_err = 0.0f;
2756 for (int i = 0; i < N; i++) {
2757     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
2758     if (err > max_err) max_err = err;
2759 }
2760 printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2761
2762 free(h_a); free(h_b); free(h_c);
2763 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
2764 }
2765
2766 static void test_histogram(int N) {
2767     srand(42);
2768     int BINS = 16;
2769     int *h_hist = (int *)calloc(BINS, sizeof(int));
2770     int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
2771     int *h_idx = (int *)malloc((size_t)N * sizeof(int));
2772     for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
2773     for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
2774
2775     int *d_idx, *d_hist;
2776     check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
2777     check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
2778     check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
2779     check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
2780
2781     dim3 block256(256);
2782     dim3 grid256((N + 255) / 256);
2783     histogram<<<grid256, block256>>>(d_idx, d_hist, N);
2784     check(cudaGetLastError(), "histogram launch");
2785     check(cudaDeviceSynchronize(), "histogram sync");
2786     check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
2787
2788     float max_err = 0.0f;
2789     for (int i = 0; i < BINS; i++) {
2790         float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
2791         if (err > max_err) max_err = err;
2792     }
2793     printf("| %-35s | %.6e | %-4s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2794
2795     free(h_hist); free(h_hist_ref); free(h_idx);
2796     cudaFree(d_idx); cudaFree(d_hist);
2797 }
2798
2799 static void test_softmax(int N) {
2800     // Use N = blockDim.x so one block processes all elements as one token.
2801     constexpr int NUM_THREADS = 256;
2802     if (N != NUM_THREADS) {
2803         printf(" OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", NUM_THREADS, N);
2804         return;
2805     }
2806 }

```

```

2809 srand(42);
2810
2811 float *h_x = (float *)malloc((size_t)N * sizeof(float));
2812 float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
2813 for (int i = 0; i < N; i++)
2814     h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]
2815
2816 // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
2817 float max_val = -FLT_MAX;
2818 for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
2819 double sum_exp = 0.0;
2820 for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
2821 for (int i = 0; i < N; i++)
2822     h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
2823
2824 float *d_x, *d_y;
2825 check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
2826 check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
2827
2828 check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
2829
2830 dim3 block(256);
2831 dim3 grid(1); // one token covering all N elements
2832 online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
2833 check(cudaGetLastError(), "softmax launch");
2834 check(cudaDeviceSynchronize(), "softmax sync");
2835
2836 float *h_y = (float *)malloc((size_t)N * sizeof(float));
2837 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
2838
2839 float max_err = 0.0f;
2840 for (int i = 0; i < N; i++) {
2841     float err = fabsf(h_y[i] - h_y_ref[i]);
2842     if (err > max_err) max_err = err;
2843 }
2844 printf("| %-35s | %.6e | %-4s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2845
2846 // ---- Safe Softmax ----
2847 safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
2848 check(cudaGetLastError(), "safe_softmax launch");
2849 check(cudaDeviceSynchronize(), "safe_softmax sync");
2850 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
2851 max_err = 0.0f;
2852 for (int i = 0; i < N; i++) {
2853     float err = fabsf(h_y[i] - h_y_ref[i]);
2854     if (err > max_err) max_err = err;
2855 }
2856 printf("| %-35s | %.6e | %-4s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2857
2858 // ---- Naive Softmax ----
2859 softmax_per_token<<<grid, block>>>(d_x, d_y, N);
2860 check(cudaGetLastError(), "naive_softmax launch");
2861 check(cudaDeviceSynchronize(), "naive_softmax sync");
2862 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
2863 max_err = 0.0f;
2864 for (int i = 0; i < N; i++) {
2865     float err = fabsf(h_y[i] - h_y_ref[i]);
2866     if (err > max_err) max_err = err;
2867 }
2868 printf("| %-35s | %.6e | %-4s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2869
2870 free(h_x); free(h_y); free(h_y_ref);
2871 cudaFree(d_x); cudaFree(d_y);
2872 }
2873
2874
2875 static void test_rms_norm(int N, int K) {
2876
2877     srand(42);
2878     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
2879     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
2880     for (int i = 0; i < N * K; i++)
2881         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2882     float g = 1.5f; // gain

```

```

2883 // CPU reference: y = (x / rms(x)) * g
2884 float epsilon = 1e-5f;
2885 for (int n = 0; n < N; n++) {
2886     double sum_sq = 0.0;
2887     for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
2888     float rms = sqrtf((float)sum_sq / (float)K + epsilon);
2889     for (int k = 0; k < K; k++)
2890         h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
2891 }
2892
2893 float *d_x, *d_y;
2894 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
2895 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
2896 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
2897
2898 dim3 block(128);
2899 dim3 grid(N);
2900 rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
2901 check(cudaGetLastError(), "rmsnorm launch");
2902 check(cudaDeviceSynchronize(), "rmsnorm sync");
2903
2904 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
2905 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
2906
2907 float max_err = 0.0f;
2908 for (int i = 0; i < N * K; i++) {
2909     float err = fabsf(h_y[i] - h_y_ref[i]);
2910     if (err > max_err) max_err = err;
2911 }
2912 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2913
2914 // ---- RMS Norm Vec4 ----
2915 dim3 block_rv4(32);
2916 rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
2917 check(cudaGetLastError(), "rmsnorm_vec4 launch");
2918 check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
2919 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
2920 max_err = 0.0f;
2921 for (int i = 0; i < N * K; i++) {
2922     float err = fabsf(h_y[i] - h_y_ref[i]);
2923     if (err > max_err) max_err = err;
2924 }
2925 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2926
2927 free(h_x); free(h_y); free(h_y_ref);
2928 cudaFree(d_x); cudaFree(d_y);
2929 }
2930
2931 static void test_layer_norm(int N, int K) {
2932
2933     srand(42);
2934     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
2935     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
2936     for (int i = 0; i < N * K; i++)
2937         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2938     float g = 1.5f, b = 0.3f; // gain and bias
2939
2940 // CPU reference: y = ((x - mean) / std) * g + b
2941 float epsilon = 1e-5f;
2942 for (int n = 0; n < N; n++) {
2943     double sum = 0.0;
2944     for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
2945     float mean = (float)sum / (float)K;
2946     double sum_sq = 0.0;
2947     for (int k = 0; k < K; k++) {
2948         float diff = h_x[n * K + k] - mean;
2949         sum_sq += (double)diff * (double)diff;
2950     }
2951     float std = sqrtf((float)sum_sq / (float)K + epsilon);
2952     for (int k = 0; k < K; k++)
2953         h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;
2954 }
2955

```

```

2957 float *d_x, *d_y;
2958 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
2959 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
2960 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
2961
2962 dim3 block(128);
2963 dim3 grid(N);
2964 layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
2965 check(cudaGetLastError(), "layernorm launch");
2966 check(cudaDeviceSynchronize(), "layernorm sync");
2967
2968 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
2969 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
2970
2971 float max_err = 0.0f;
2972 for (int i = 0; i < N * K; i++) {
2973     float err = fabsf(h_y[i] - h_y_ref[i]);
2974     if (err > max_err) max_err = err;
2975 }
2976 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2977
2978 // ---- Layer Norm Vec4 ----
2979 dim3 block_lv4(32);
2980 layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
2981 check(cudaGetLastError(), "layernorm_vec4 launch");
2982 check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
2983 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
2984 max_err = 0.0f;
2985 for (int i = 0; i < N * K; i++) {
2986     float err = fabsf(h_y[i] - h_y_ref[i]);
2987     if (err > max_err) max_err = err;
2988 }
2989 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2990
2991 free(h_x); free(h_y); free(h_y_ref);
2992 cudaFree(d_x); cudaFree(d_y);
2993 }
2994
2995 static void test_rope(int seq_len, int N) {
2996
2997     int total_pairs = seq_len * N;
2998     int total_elems = total_pairs * 2;
2999     size_t size = (size_t)total_elems * sizeof(float);
3000
3001     float *h_x = (float *)malloc(size);
3002     float *h_y_ref = (float *)malloc(size);
3003
3004     srand(42);
3005     for (int i = 0; i < total_elems; i++)
3006         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3007
3008     // CPU reference: 2D rotation for each pair
3009     for (int idx = 0; idx < total_pairs; idx++) {
3010         int token_pos = idx / N;
3011         int token_idx = idx % N;
3012         float x1 = h_x[idx * 2];
3013         float x2 = h_x[idx * 2 + 1];
3014         float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
3015         float angle = (float)token_pos * theta;
3016         float cos_v = cosf(angle);
3017         float sin_v = sinf(angle);
3018         h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
3019         h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
3020     }
3021
3022     float *d_x, *d_y;
3023     check(cudaMalloc(&d_x, size), "rope alloc X");
3024     check(cudaMalloc(&d_y, size), "rope alloc Y");
3025     check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
3026
3027     dim3 block(256);
3028     dim3 grid((total_pairs + 255) / 256);

```

```

3031 rope<<<grid, block>>>(d_x, d_y, seq_len, N);
3032 check(cudaGetLastError(), "rope launch");
3033 check(cudaDeviceSynchronize(), "rope sync");
3034
3035 float *h_y = (float *)malloc(size);
3036 check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
3037
3038 float max_err = 0.0f;
3039 for (int i = 0; i < total_elems; i++) {
3040     float err = fabsf(h_y[i] - h_y_ref[i]);
3041     if (err > max_err) max_err = err;
3042 }
3043 printf("| %-35s | %.6e | %-4s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3044
3045 free(h_x);
3046 free(h_y);
3047 free(h_y_ref);
3048 cudaFree(d_x);
3049 cudaFree(d_y);
3050 }
3051
3052
3053 static void test_mat_transpose(int row, int col) {
3054
3055     size_t size_in = (size_t)row * col * sizeof(float);
3056     size_t size_out = (size_t)col * row * sizeof(float);
3057
3058     float *h_x = (float *)malloc(size_in);
3059     float *h_y_ref = (float *)malloc(size_out);
3060
3061     srand(42);
3062     for (int i = 0; i < row * col; i++)
3063         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3064
3065     // CPU reference: y[j][i] = x[i][j] (row-major)
3066     for (int i = 0; i < row; i++)
3067         for (int j = 0; j < col; j++)
3068             h_y_ref[j * row + i] = h_x[i * col + j];
3069
3070     float *d_x, *d_y;
3071     check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
3072     check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
3073     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
3074
3075     dim3 block(16, 16);
3076     dim3 grid((col + 15) / 16, (row + 15) / 16);
3077     mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
3078     check(cudaGetLastError(), "mattrans launch");
3079     check(cudaDeviceSynchronize(), "mattrans sync");
3080
3081     float *h_y = (float *)malloc(size_out);
3082     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
3083
3084     float max_err = 0.0f;
3085     for (int i = 0; i < col * row; i++) {
3086         float err = fabsf(h_y[i] - h_y_ref[i]);
3087         if (err > max_err) max_err = err;
3088     }
3089     printf("| %-35s | %.6e | %-4s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
3090
3091     free(h_x);
3092     free(h_y);
3093     free(h_y_ref);
3094     cudaFree(d_x);
3095     cudaFree(d_y);
3096 }
3097
3098
3099 static void test_mat_transpose_padded(int row, int col) {
3100
3101     size_t size_in = (size_t)row * col * sizeof(float);
3102     size_t size_out = (size_t)col * row * sizeof(float);
3103
3104     float *h_x = (float *)malloc(size_in);

```

```

3105 float *h_y_ref = (float *)malloc(size_out);
3106
3107 srand(42);
3108 for (int i = 0; i < row * col; i++)
3109     h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3110
3111 // CPU reference: y[j][i] = x[i][j] (row-major)
3112 for (int i = 0; i < row; i++)
3113     for (int j = 0; j < col; j++)
3114         h_y_ref[j * row + i] = h_x[i * col + j];
3115
3116 float *d_x, *d_y;
3117 check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
3118 check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
3119 check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
3120
3121 dim3 block(16, 16);
3122 dim3 grid((col + 15) / 16, (row + 63) / 64);
3123 mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
3124 check(cudaGetLastError(), "mattrans_padded launch");
3125 check(cudaDeviceSynchronize(), "mattrans_padded sync");
3126
3127 float *h_y = (float *)malloc(size_out);
3128 check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
3129
3130 float max_err = 0.0f;
3131 for (int i = 0; i < col * row; i++) {
3132     float err = fabsf(h_y[i] - h_y_ref[i]);
3133     if (err > max_err) max_err = err;
3134 }
3135 printf("| %-35s | %.6e | %-4s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
3136
3137 free(h_x);
3138 free(h_y);
3139 free(h_y_ref);
3140 cudaFree(d_x);
3141 cudaFree(d_y);
3142 }
3143
3144
3145 static void test_sgemv(int M, int K) {
3146
3147     srand(42);
3148     float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
3149     float *h_x = (float *)malloc((size_t)K * sizeof(float));
3150     float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
3151     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3152     for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3153
3154     // CPU reference: y[m] = sum_k A[m,k] * x[k]
3155     for (int m = 0; m < M; m++) {
3156         double sum = 0.0;
3157         for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
3158         h_y_ref[m] = (float)sum;
3159     }
3160
3161     float *d_a, *d_x, *d_y;
3162     check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
3163     check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
3164     check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
3165
3166     check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
3167     check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
3168
3169     dim3 block(32, 4);
3170     dim3 grid((M + 3) / 4);
3171     sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
3172     check(cudaGetLastError(), "sgemv launch");
3173     check(cudaDeviceSynchronize(), "sgemv sync");
3174
3175     float *h_y = (float *)malloc((size_t)M * sizeof(float));
3176     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
3177
3178     float max_err = 0.0f;

```

```

3179     for (int m = 0; m < M; m++) {
3180         float err = fabsf(h_y[m] - h_y_ref[m]);
3181         if (err > max_err) max_err = err;
3182     }
3183     printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3184
3185     // ---- SGEMV K32 ----
3186     check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
3187     sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
3188     check(cudaGetLastError(), "sgemv_k32 launch");
3189     check(cudaDeviceSynchronize(), "sgemv_k32 sync");
3190
3191     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
3192
3193     max_err = 0.0f;
3194     for (int m = 0; m < M; m++) {
3195         float err = fabsf(h_y[m] - h_y_ref[m]);
3196         if (err > max_err) max_err = err;
3197     }
3198     printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3199
3200     // ---- SGEMV K16 ----
3201     free(h_a); free(h_x); free(h_y); free(h_y_ref);
3202     cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
3203
3204     int K16 = 16;
3205     h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
3206     h_x = (float *)malloc((size_t)K16 * sizeof(float));
3207     h_y_ref = (float *)malloc((size_t)M * sizeof(float));
3208     for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3209     for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3210
3211     for (int m = 0; m < M; m++) {
3212         double sum = 0.0;
3213         for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
3214         h_y_ref[m] = (float)sum;
3215     }
3216
3217     check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
3218     check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
3219     check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
3220
3221     check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
3222     check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
3223
3224     dim3 grid_k16((M + 7) / 8);
3225     sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
3226     check(cudaGetLastError(), "sgemv_k16 launch");
3227     check(cudaDeviceSynchronize(), "sgemv_k16 sync");
3228
3229     h_y = (float *)malloc((size_t)M * sizeof(float));
3230     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
3231
3232     max_err = 0.0f;
3233     for (int m = 0; m < M; m++) {
3234         float err = fabsf(h_y[m] - h_y_ref[m]);
3235         if (err > max_err) max_err = err;
3236     }
3237     printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3238
3239     free(h_a); free(h_x); free(h_y); free(h_y_ref);
3240     cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
3241 }
3242
3243
3244 static void test_sgemm(int M, int N, int K) {
3245
3246     size_t size_a = (size_t)M * K * sizeof(float);
3247     size_t size_b = (size_t)K * N * sizeof(float);
3248     size_t size_c = (size_t)M * N * sizeof(float);
3249
3250     float *h_a = (float *)malloc(size_a);
3251     float *h_b = (float *)malloc(size_b);
3252     float *h_c_ref = (float *)malloc(size_c);

```



```

3253 srand(42);
3254
3255 for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3256 for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3257
3258 float *d_a, *d_b, *d_c;
3259 check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
3260 check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
3261 check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
3262
3263 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
3264 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
3265
3266 // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
3267 cublasHandle_t handle;
3268 cublasCreate(&handle);
3269 float alpha = 1.0f, beta = 0.0f;
3270 cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
3271             &alpha, d_b, N, d_a, K, &beta, d_c, N);
3272 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
3273
3274 // Kernel
3275 cudaEvent_t start, stop;
3276 cudaEventCreate(&start);
3277 cudaEventCreate(&stop);
3278
3279 dim3 block(32, 32);
3280 dim3 grid((N + 31) / 32, (M + 31) / 32);
3281 sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
3282 check(cudaGetLastError(), "sgemm launch");
3283 check(cudaDeviceSynchronize(), "sgemm sync");
3284
3285 float *h_c = (float *)malloc(size_c);
3286 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
3287
3288 // Verify
3289 float max_err = 0.0f;
3290 for (int i = 0; i < M * N; i++) {
3291     float err = fabsf(h_c[i] - h_c_ref[i]);
3292     if (err > max_err) max_err = err;
3293 }
3294 printf("| %-35s | %.6e | %-4s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3295
3296 // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
3297
3298 dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
3299 check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
3300 sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
3301 check(cudaGetLastError(), "sgemm_vec4 launch");
3302 check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
3303
3304 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
3305
3306 max_err = 0.0f;
3307 for (int i = 0; i < M * N; i++) {
3308     float err = fabsf(h_c[i] - h_c_ref[i]);
3309     if (err > max_err) max_err = err;
3310 }
3311 printf("| %-35s | %.6e | %-4s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3312
3313 free(h_a); free(h_b); free(h_c); free(h_c_ref);
3314 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
3315 cublasDestroy(handle);
3316 cudaEventDestroy(start);
3317 cudaEventDestroy(stop);
3318 }
3319
3320
3321 static void test_hgemm_mma(int M, int N, int K) {
3322
3323     size_t size_a = (size_t)M * K * sizeof(half);
3324     size_t size_b = (size_t)K * N * sizeof(half);
3325     size_t size_c = (size_t)M * N * sizeof(half);
3326

```

```

3327 half *h_a = (half *)malloc(size_a);
3328 half *h_b = (half *)malloc(size_b);
3329 half *h_c_ref = (half *)malloc(size_c);
3330
3331 srand(42);
3332 for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3333 for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3334
3335 // Kernel expects B^T [N×K] row-major layout (TN layout convention).
3336 // We store h_b as B [K×N] row-major for cuBLAS, then create B^T for kernel.
3337 size_t size_b_t = (size_t)N * K * sizeof(half);
3338 half *h_b_t = (half *)malloc(size_b_t);
3339 for (int n = 0; n < N; n++)
3340     for (int k = 0; k < K; k++)
3341         h_b_t[n * K + k] = h_b[k * N + n];
3342
3343 half *d_a, *d_b, *d_b_t, *d_c;
3344 check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
3345 check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
3346 check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
3347 check(cudaMalloc(&d_c, size_c), "hgemm alloc C");
3348
3349 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
3350 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
3351 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");
3352
3353 // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
3354 // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
3355 cublasHandle_t handle;
3356 cublasCreate(&handle);
3357 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
3358 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
3359             &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
3360             &beta_h, d_c, CUDA_R_16F, N,
3361             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
3362 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");
3363
3364 // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
3365 cudaEvent_t start, stop;
3366 cudaEventCreate(&start);
3367 cudaEventCreate(&stop);
3368
3369 constexpr int BM = 128, BN = 128, BK = 16, K_STAGE = 3;
3370 size_t smem_bytes = K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 24576
3371 dim3 block(256);
3372 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
3373 hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
3374 check(cudaGetLastError(), "hgemm launch");
3375 check(cudaDeviceSynchronize(), "hgemm sync");
3376
3377 half *h_c = (half *)malloc(size_c);
3378 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");
3379
3380 // Verify
3381 float max_err = 0.0f;
3382 for (int i = 0; i < M * N; i++) {
3383     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
3384     if (err > max_err) max_err = err;
3385 }
3386 printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL");
3387
3388 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
3389 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
3390 cublasDestroy(handle);
3391 cudaEventDestroy(start);
3392 cudaEventDestroy(stop);
3393 }
3394
3395
3396 static void test_flash_attn(int seqlen, int head_dim) {
3397     // FlashAttention-2 with split-Q, MMA m16n8k16
3398     int B = 1, H = 8;
3399
3400     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);

```

```

3401 srand(42);
3402
3403 half *h_q = (half *)malloc(sz);
3404 half *h_k = (half *)malloc(sz);
3405 half *h_v = (half *)malloc(sz);
3406 for (int i = 0; i < B * H * seqlen * head_dim; i++) {
3407     h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3408     h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3409     h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3410 }
3411
3412 // CPU reference in FP32: 0 = softmax(Q @ K^T / sqrt(d)) @ V
3413 float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
3414 float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
3415 float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
3416 float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
3417 int count = B * H * seqlen * head_dim;
3418 for (int i = 0; i < count; i++) {
3419     ref_q[i] = __half2float(h_q[i]);
3420     ref_k[i] = __half2float(h_k[i]);
3421     ref_v[i] = __half2float(h_v[i]);
3422 }
3423
3424 float scale = 1.0f / sqrtf((float)head_dim);
3425 for (int bi = 0; bi < B * H; bi++) {
3426     for (int qi = 0; qi < seqlen; qi++) {
3427         // S[qi, kj] = Q[qi, :] @ K[kj, :]^T * scale
3428         float smax = -INFINITY;
3429         float *S = (float *)malloc((size_t)seqlen * sizeof(float));
3430         for (int kj = 0; kj < seqlen; kj++) {
3431             float s = 0.0f;
3432             for (int d = 0; d < head_dim; d++)
3433                 s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
3434                     ref_k[bi * seqlen * head_dim + kj * head_dim + d];
3435             S[kj] = s * scale;
3436             if (S[kj] > smax) smax = S[kj];
3437         }
3438         // softmax
3439         double sum_exp = 0.0;
3440         for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
3441         float inv_sum = 1.0f / (float)sum_exp;
3442         // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
3443         for (int d = 0; d < head_dim; d++) {
3444             double o_acc = 0.0;
3445             for (int kj = 0; kj < seqlen; kj++)
3446                 o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
3447                     ref_v[bi * seqlen * head_dim + kj * head_dim + d];
3448             ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
3449         }
3450         free(S);
3451     }
3452 }
3453
3454 half *d_q, *d_k, *d_v, *d_o;
3455 check(cudaMalloc(&d_q, sz), "fa alloc Q");
3456 check(cudaMalloc(&d_k, sz), "fa alloc K");
3457 check(cudaMalloc(&d_v, sz), "fa alloc V");
3458 check(cudaMalloc(&d_o, sz), "fa alloc O");
3459 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
3460 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
3461 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
3462
3463 // Template params for kHeadDim=64, kStage=2
3464 constexpr int kHeadDim = 64, kStageV = 2, kPadV = 8;
3465 constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
3466 constexpr int kMmaTileSeqLenQ = 4;
3467 constexpr int kMmaTileSeqLenK = 1;
3468 constexpr int kMmaTileSeqLenP = 4;
3469 constexpr int kMmaTileHeadDimV = 1;
3470 constexpr int kWarpTileSeqLenQ = 1;
3471 constexpr int kWarpTileSeqLenK = 8;
3472 constexpr int kWarpTileSeqLenP = 1;
3473 constexpr int kWarpTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
3474

```

```

3475 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kWarpTileSeqLenQ;
3476 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kWarpTileSeqLenK;
3477 size_t smem_bytes = (Br * (kHeadDim + kPadV) +
3478                     kStageV * Bc * (kHeadDim + kPadV) +
3479                     Bc * (kHeadDim + kPadV)) * sizeof(half);
3480
3481 dim3 block(128);
3482 dim3 grid((seqlen + Br - 1) / Br, B * H);
3483
3484 flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK,
3485 kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV,
3486 kWarpTileSeqLenQ, kWarpTileSeqLenK, kWarpTileSeqLenP, kWarpTileHeadDimV,
3487 kStageV, kPadV>
3488 <<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
3489 check(cudaGetLastError(), "fa launch");
3490 check(cudaDeviceSynchronize(), "fa sync");
3491
3492 half *h_o = (half *)malloc(sz);
3493 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
3494
3495 float max_err = 0.0f;
3496 for (int i = 0; i < count; i++) {
3497     float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
3498     if (err > max_err) max_err = err;
3499 }
3500 printf("| %-35s | %.6e | %-4s |\n", "FlashAttn-SplitQ", max_err, max_err < 1e-1f ? "PASS" : "FAIL");
3501
3502 free(h_q); free(h_k); free(h_v); free(h_o);
3503 free(ref_q); free(ref_k); free(ref_v); free(ref_o);
3504 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
3505 }
3506
3507 int main(int argc, char *argv[]) {
3508     int M = 1024, N = 1024, K = 1024;
3509     if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
3510
3511     printf("=== notes-v2.cu verification harness ===\n");
3512     printf("| %-35s | %-12s | %-4s |\n", "Kernel", "Max Err", "Pass");
3513     printf("|-----|-----|-----|\n");
3514
3515     test_block_reduce(N);
3516     test_dot(N);
3517     test_relu(1024);
3518     test_elementwise(1024);
3519     test_histogram(1024);
3520     test_softmax(256); // softmax kernel requires N == blockDim.x
3521     test_rms_norm(8, 128);
3522     test_layer_norm(8, 128);
3523     test_rope(8, 128);
3524     test_mat_transpose(256, 256);
3525     test_mat_transpose_padded(256, 256);
3526     test_sgemv(256, 128);
3527     test_sgemm(M, N, K);
3528     test_hgemm_mma(M, N, K);
3529     test_flash_attn(1024, 64);
3530
3531     printf("=== All tests done ===\n");
3532     return 0;
3533 }
3534
3535 // =====
3536 // Quick build & run reference
3537 // =====
3538 // # sm_89 单独编译 + 运行:
3539 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
3540 //
3541 // # sm_90a 单独编译 (运行需要 H800 Hopper GPU) :
3542 // nvcc -std=c++20 -O2 -arch=sm_90a -lcublas -lcuda notes-v2.cu -o notes_v2_sm90.bin

```